TU Clausthal

Master's Thesis

by

Yu Zhang

Explain Reality: Grounded Procedural Tutoring with Foundation Models and Visual Fact Extraction in Immersive Simulation

1st Supervisor:	**Prof. Dr. Christian Bartelt**
2nd Supervisor:	**Prof. Dr. Rüdiger Ehlers**

11th May 2026

Institute for Software and Systems Engineering
Clausthal University of Technology

Declaration of Authorship

I have read and understood the guidelines of the Technical University of Clausthal and those published in the ISSE bulletin, including those regarding the use of literature and other sources. I confirm that I have prepared this thesis independently by myself. Any information taken from other sources and being reproduced in this thesis is clearly referenced.

In terms of the general examination regulations, this work has not yet been submitted to any other examination division.

I hereby agree that my master's thesis may be exhibited in the institute's library and kept for inspection.

Clausthal-Zellerfeld, 11th May 2026	
Location, Date	Yu Zhang

Abstract

Procedural learning in high-fidelity simulation environments presents distinct challenges: learners must execute fixed, ordered sequences of actions while verifying intermediate system states, recovering from errors, and maintaining situational awareness—all without the continuous presence of an expert instructor. Traditional training aids such as video walkthroughs and printed checklists break the continuity of in-simulator practice, while ungrounded language model assistance risks providing fluent but factually incorrect guidance. This thesis presents SimTutor, a telemetry-grounded tutoring runtime that combines procedure-pack-driven state tracking, retrieval-augmented language model help generation, and a fine-tuned vision-language model (VLM) for structured visual fact extraction from cockpit displays. The system enforces explicit evidence grounding: every overlay target and step recommendation must be traceable to a telemetry variable, a gating rule, a retrieved document snippet, or a visual fact observation. The training scenario is the F/A-18C cold-start procedure in DCS World, comprising 25 steps across six operational phases, with inter-step dependencies that make it representative of sequence-sensitive, state-dependent procedural tasks. The VLM adaptation pipeline—screenshot capture, pre-labeling, human review, bilingual supervised fine-tuning, LoRA adaptation, and automated benchmarking—is applied to two backbone architectures. On the primary 13-fact ontology, a Qwen3.5-9B LoRA adapter trained on 928 bilingual rows improves fact accuracy from 0.80–0.87 (base model) to 0.99 on two independent holdout sets with verified zero training overlap, while reducing critical false positives by 64–89%. An identical data recipe applied to Gemma-4-31B confirms that the improvement generalises across backbones. The thesis delivers a working desktop runtime with automated tests, benchmark artifacts, and replay infrastructure. VR runtime integration is implemented in code (VR_MIRROR configuration, tutor text display, composite-panel capture); controlled human-subject evaluation is the remaining planned work. The current system provides the technical baseline on which that evaluation can be built.

Zusammenfassung

Prozedurales Lernen in hochrealistischen Simulationsumgebungen stellt besondere Herausforderungen: Lernende müssen feste, geordnete Handlungsabfolgen ausführen, während sie Zwischenzustände des Systems überprüfen, Fehler korrigieren und das Situationsbewusstsein aufrechterhalten—und dies ohne die ständige Anwesenheit eines erfahrenen Ausbilders. Herkömmliche Trainingshilfen wie Videoanleitungen und gedruckte Checklisten unterbrechen den Übungsfluss im Simulator, während nicht zustandsgebundene Sprachmodell-Assistenz Gefahr läuft, flüssige, aber faktisch inkorrekte Anweisungen zu geben. Diese Arbeit stellt SimTutor vor, eine telemetriegestützte Tutoring-Laufzeitumgebung, die paketbasierte Prozedurzustandsverfolgung, Retrieval-Augmented-LLM-Hilfegenerierung und ein feinabgestimmtes Vision-Language-Model (VLM) zur strukturierten Extraktion visueller Fakten aus Cockpit-Anzeigen kombiniert. Das System erzwingt eine explizite Evidenzgrundlage: Jede hervorgehobene Cockpit-Referenz und jede Schrittempfehlung muss auf eine Telemetrie-Variable, eine Gating-Regel, einen abgerufenen Dokumentausschnitt oder eine visuelle Faktbeobachtung zurückführbar sein. Als Trainingsszenario dient der Kaltstart der F/A-18C in DCS World mit 25 Schritten in sechs Betriebsphasen, deren Abhängigkeiten untereinander sie repräsentativ für zustandsabhängige prozedurale Aufgaben machen. Die VLM-Adaptationspipeline—Screenshot-Erfassung, Vorlabeling, menschliche Überprüfung, bilingualer SFT-Export, LoRA-Feinabstimmung und automatisiertes Benchmarking—wird auf zwei Backbone-Architekturen angewendet. Auf der primären 13-Fact-Ontologie verbessert ein Qwen3.5-9B LoRA-Adapter (928 bilinguale Trainingszeilen) die Fact-Genauigkeit von 0,80–0,87 (Basismodell) auf 0,99 auf zwei unabhängigen Holdout-Sets mit verifizierter Null-Überlappung zum Training und reduziert kritische False-Positives um 64–89%. Die identische Datenrezeptur, auf Gemma-4-31B angewendet, bestätigt die Übertragbarkeit der Verbesserung über Backbones hinweg. Die Arbeit liefert eine funktionsfähige Desktop-Laufzeitumgebung mit automatisierten Tests, Benchmark-Artefakten und Replay-Infrastruktur. Die VR-Laufzeitintegration ist im Code implementiert (VR_MIRROR-Konfiguration, Tutor-Text-Anzeige, Composite-Panel-Erfassung); die kontrollierte Human-Subject-Evaluation ist die verbleibende ge-

plante Arbeit. Das aktuelle System stellt die technische Basis dar, auf der diese Evaluation aufgebaut werden kann.

Acknowledgements

Contents

List of Figures

List of Tables

1 Introduction

This thesis describes the design, implementation, and technical evaluation of SimTutor, a telemetry-grounded tutoring runtime that combines procedure-pack-driven state tracking, retrieval-augmented language model help generation, and fine-tuned vision-language model (VLM) visual fact extraction to support procedural learning in a high-fidelity flight simulator. The work is motivated by the challenges of teaching complex, sequence-sensitive procedures—exemplified by the F/A-18C cold-start task in DCS World—and by the gap between the original research vision of a virtual-reality (VR) grounded tutoring system and the technical baseline that has been completed to date. This chapter defines the research problem, traces the evolution from proposal to current implementation, enumerates the contributions of the present work, and outlines the structure of the remainder of the thesis.

1.1 Motivation

Procedural learning—the acquisition of fixed, ordered sequences of actions that must be executed correctly and in the right context—presents distinct challenges that distinguish it from declarative or conceptual learning. A learner must not only memorise the steps but also recognise when each step applies, verify that its preconditions are satisfied, confirm its completion before advancing, and recover from errors without propagating mistakes through the remainder of the procedure. In high-fidelity simulation environments, these challenges are compounded by the complexity of the cockpit or workspace, the density of instrument information, and the cognitive load of maintaining situational awareness while executing an unfamiliar sequence.

Traditional training aids—video walkthroughs, printed checklists, instructor-led demonstrations—are effective for initial familiarisation but break the continuity of in-simulator practice. A learner who pauses to consult a manual or replay a video segment must context-switch away from the cockpit, losing immersion and, in some cases, the transient simulator state that motivated the consultation in the first place. Conversely, ungrounded text-based

assistance from a large language model (LLM) may provide fluent but factually incorrect guidance because the model has no access to the current simulator state. Between these extremes lies the need for *grounded, context-aware* tutoring: a system that observes the learner's actions through live simulator telemetry, retrieves relevant documentation, extracts visual evidence from the cockpit where telemetry is insufficient, and delivers concise, verifiable guidance without disrupting the training flow.

The F/A-18C cold-start procedure in DCS World serves as the training scenario for this thesis. The procedure comprises 25 steps (S01–S25) grouped into six phases (P1–P6), from battery power-up through engine start, avionics configuration, flight-control checks, and taxi preparation (see §2.1 for a detailed description). Each step carries explicit preconditions that depend on the completion of earlier steps and on the satisfaction of specific telemetry conditions: for example, the left engine crank (S10) requires right-engine parameters—RPM, temperature, fuel flow, nozzle position, and oil pressure—to fall within their nominal ranges before the left engine is engaged. These inter-step dependencies make the cold-start task a representative instance of the broader class of sequence-sensitive, state-dependent procedural tasks that characterise simulator-based training in aviation, maritime operations, industrial maintenance, and medical procedure instruction. TODO:CITE — intelligent tutoring systems and procedural learning literature.

The central premise of this work is that a tutoring system can be made substantially more reliable and auditable by grounding every actionable output in explicit evidence: a telemetry variable, a gating-rule evaluation, a retrieved document snippet, or a visual fact observation. This evidence-grounding constraint is not merely an architectural preference but a safety requirement: in the context of cockpit procedures, an incorrectly highlighted control or a misidentified procedure state could confuse or mislead the learner. The SimTutor system described in this thesis instantiates this principle in a working runtime with automated tests, benchmark artifacts, and replay infrastructure.

1.2 Thesis Vision and Scope

The original research vision for this thesis, as articulated in the research proposal, centred on a VR-first grounded tutoring system operating on the Meta Quest 3 with DCS-VR [**?**]. The planned system would receive learner interactions through gaze, hand tracking, and voice input within the VR cockpit; retrieve relevant manual and checklist excerpts; generate concise, citation-backed step guidance through a retrieval-augmented

LLM; and render the guidance as in-headset step cards or overlay annotations registered to cockpit controls. The proposal further specified a three-condition human-subject evaluation comparing a video-and-notes baseline, an ungrounded text-only LLM, and the grounded tutoring system, with metrics including completion rate, procedural error rate, task time, NASA-TLX workload scores, pre/post knowledge quizzes, and retention and transfer tests administered after a 2–4 week interval.

This vision defines the long-term research agenda of the thesis. However, the work completed to date represents a technical baseline rather than a realisation of the full VR tutoring system validated through human-subject experiments. The current implementation supports both desktop/DCS and VR (Quest 3) configurations, uses cockpit overlay highlighting as its primary output modality, and has been validated through automated benchmarks rather than human-subject experiments. The human-subject evaluation remains the planned next phase (see Chapter 6, Section 6.2).

We document this gap explicitly at the outset to establish a clear boundary between the original research vision and the completed technical contributions. The thesis should be read as: a system and methodology contribution that lays the technical foundation—runtime, instrumentation, VLM adaptation pipeline, benchmark infrastructure—on which the human-subject evaluation can be built.

1.3 Current Implementation Scope

What has been built differs from the proposal in several important respects, and these differences are themselves a source of the thesis's contributions. A detailed comparison appears in Section 3.10; we summarise the key differences here to orient the reader.

Platform. The current system supports both desktop/DCS and VR (Quest 3) configurations. VR support is provided through VR_MIRROR and VR_allow_MFD_out_of_HMD configurations, tutor text UDP injection into the DCS in-game text area, and composite-panel capture compatibility validation in VR mode. On desktop, displays are standard monitors; input is through keyboard and HOTAS (hands-on-throttle-and-stick); output is rendered as cockpit overlay highlights on the DCS screen. The VR runtime code path has been implemented but has not yet been validated through human-subject experiments.

Telemetry grounding. The current implementation includes a substantially more elaborate telemetry subsystem than the proposal anticipated: raw DCS-BIOS UDP ingestion, variable resolution with source-missing tracking, delta sanitisation (threshold, debounce, and per-variable filtering), and delta aggregation for prompt-safe compression. These additions were driven by the practical realities of working with noisy, high-frequency simulator telemetry.

VLM visual fact extraction. The most significant addition relative to the proposal is the VLM visual fact extraction pipeline. The original proposal envisioned a text-only RAG+LLM tutor with read-only simulator state as the sole grounding signal. During implementation, it became clear that several critical procedure steps (S08, S15, S18) require visual confirmation of display-page content that is not exported by DCS-BIOS. The VLM pipeline was added to address this gap and evolved into a self-contained research contribution spanning structured visual fact ontology design, dataset curation with human review, bilingual LoRA fine-tuning, and automated benchmark evaluation across two model backbones.

Procedure pack and infrastructure. The current system organises procedure definitions, gating rules, telemetry mappings, error taxonomies, and vision fact ontologies as declarative YAML configuration (a *procedure pack*) rather than encoding them implicitly in prompts or code. It also includes an event logging and replay infrastructure, an automated scoring engine, and a deterministic fallback path that operates when no model is available or when model output fails validation. None of these elements were specified in the original proposal.

These differences do not invalidate the proposal; they reflect the engineering reality that a grounded tutoring runtime requires substantial supporting infrastructure before it can be deployed in a VR human-subject study. The current system provides precisely this infrastructure.

1.4 Contributions

This thesis makes the following contributions:

1. **A pack-driven, telemetry-grounded tutoring runtime architecture.** We present the design and implementation of a tutoring runtime organised in a ports-and-adapters (hexagonal) architecture that separates simulator-agnostic core logic

(procedure tracking, gating, scoring, knowledge retrieval) from simulator-specific adapters (DCS-BIOS telemetry, overlay rendering, vision capture). The system supports live help cycles in which the runtime assembles a structured context from normalised telemetry variables, delta summaries, candidate procedure steps, a deterministic step hint, an overlay-target allowlist, retrieved knowledge snippets, and visual fact observations; passes this context to an LLM for structured help response generation; validates the response against schema, allowlist, and evidence-grounding constraints; and executes the validated overlay actions while logging every stage for downstream replay and analysis. A deterministic fallback path ensures the system remains functional when no model is available or when model output fails validation. This contribution is described in Chapters 3, 4, and 5.

2. **A structured visual fact ontology and a complete VLM data, adaptation, and benchmark pipeline.** We define a 13-fact visual ontology that decomposes cockpit display-page state into discrete, auditable propositions spanning three display regions (Left DDI, Right DDI, AMPCD). The ontology evolved across two generations—from an 8-fact v1 early baseline to the current runtime-aligned 13-fact v2—with lessons learned about abstraction-level mixing and false-positive risk. We implement a complete adaptation pipeline: screenshot capture in DCS, pre-labeling by a large VLM, human review in Label Studio, bilingual (English and Chinese) supervised fine-tuning export, LoRA fine-tuning using Unsloth, PEFT, and TRL, and automated base-vs-LoRA benchmarking on independent holdout sets with verified zero training overlap. Every artifact in this pipeline—raw captures, prelabels, human-reviewed labels, SFT datasets, training summaries, benchmark metrics, and per-fact confusion charts—is stored in the repository and can be independently inspected and reproduced. This contribution is described in Chapters 3 and 4.

3. **Benchmark results demonstrating that small-data LoRA adaptation substantially improves cockpit VLM fact extraction accuracy on independent holdout sets.** Under the 13-fact v2 ontology, a Qwen3.5-9B LoRA adapter trained on 928 bilingual rows (Run-003 + Run-005×2) achieves a fact accuracy of 0.9908 (base: 0.8677) on the Run-002 newfacts holdout (50 samples) and 0.9931 (base: 0.8038) on the Run-004 random holdout (100 samples). Sample exact match rates rise from 0.10 to 0.88 on Run-002 and from 0.03 to 0.92 on Run-004. The critical false positive count—errors that carry direct downstream

risk of prematurely advancing the procedure—drops by 64% on Run-002 and by 89% on Run-004. A per-fact error analysis identifies completion-fact false positives as the primary residual failure mode. These results are presented in Chapter 6, Section 6.1.

4. **A backbone comparison showing that the data recipe generalises across model architectures.** The identical data recipe (Run-003 + Run-005×2, 928 rows, bilingual) was applied to a second VLM backbone, Gemma-4-31B. The Gemma LoRA adapter achieves a fact accuracy of 0.9492 (base: 0.8169) on the Run-002 newfacts holdout and 0.9715 (base: 0.8146) on the Run-004 random holdout, with zero critical false positives on Run-002. While the Qwen adapter achieves higher absolute accuracy and recall, the Gemma results confirm that the data recipe—rather than a backbone-specific artefact—drives the improvement, and reveal backbone-level differences in precision–recall trade-off that are relevant for safety-critical deployment. These results are presented in Chapter 6, Section 6.1.4.

5. **A planned evaluation framework for future VR human-subject studies.** We specify, at the design level, a controlled between-subjects evaluation comparing video-and-notes, ungrounded text-only LLM, and grounded tutoring conditions, with planned metrics spanning immediate performance (completion rate, error rate, task time), workload (NASA-TLX), learning (pre/post quizzes), retention (2–4 week delayed post-test), and transfer (variant procedure). We identify the runtime data collection upgrades required to transition from the current replay infrastructure to a human-subject-ready data pipeline and catalogue the known threats to validity at the design stage. This planned evaluation is described in Chapter 6, Section 6.2, and represents the next phase of the research programme.

Contributions 1–4 are supported by verified claims and benchmark artifacts currently present in the repository. Contribution 5 is a methodological and design contribution that documents the evaluation pathway; it does not report human-subject data, and its stated metrics and comparisons remain planned rather than completed.

1.5 Thesis Structure

The remainder of the thesis is organised as follows.

Chapter 2 provides the domain and technical background necessary to understand the system. It describes the F/A-18C cold-start procedure in detail, introduces the

visual fact ontology and its evolution from 8-fact v1 to 13-fact v2, and surveys related work across intelligent tutoring systems, simulator-based training, rule-based and state-aware tutoring, VLM adaptation for structured outputs, and explainable AI assistance in AR/VR contexts.

Chapter 3 presents the design of SimTutor. It derives design goals from the problem statement, describes the ports-and-adapters architecture, traces the runtime data flow through a complete help cycle, and details the design of the telemetry processing pipeline, the procedure and gating rule engines, the simulator adapter layer, the event logging and replay infrastructure, and the VLM adaptation component. The chapter closes with a systematic comparison to the original thesis proposal, documenting how the design evolved during implementation.

Chapter 4 describes the methodological framework for the VLM adaptation pipeline. It covers dataset curation (Run-001 through Run-005, ontology versions, label distributions, holdout construction with verified zero overlap), the capture–prelabel–review–export–train–benchmark pipeline, LoRA fine-tuning configuration for Qwen3.5-9B and Gemma-4-31B, and the benchmark protocol including metric definitions and contamination controls.

Chapter 5 documents implementation-level details of the technology stack, core domain logic modules, telemetry and DCS integration, structured help generation, VLM visual fact extraction, replay and logging infrastructure, and CLI and runtime entrypoints.

Chapter 6 is divided into two parts. Part A reports the current technical results: the dataset and training summary, the 8-fact v1 baseline, the primary 13-fact v2 Qwen benchmark results, the Gemma backbone comparison, a per-fact error analysis, and an assessment of runtime infrastructure readiness. Part B describes the planned human-subject evaluation: the original evaluation vision, planned metrics, required infrastructure upgrades, the proposed experiment design, and a catalogue of known threats to validity. All statements in Part B represent future work.

Chapter 7 interprets the results and their limitations. It examines why the project shifted from a VR-first to a telemetry/VLM-heavy implementation, clarifies what the current benchmarks do and do not prove, analyses the ontology evolution from 8-fact to 13-fact, argues why human-subject evidence remains essential, and identifies the risks of interpreting benchmark success as learning success.

Chapter 8 summarises the completed work, restates the contributions, and outlines directions for future work including VR runtime integration, human-subject evaluation,

ontology extension to other procedures, multi-frame temporal reasoning, and online adaptation.

2 Background and Related Work

This chapter provides the domain and technical background necessary to understand the design of the SimTutor system, followed by a survey of related work across several intersecting research areas. Section 2.1 introduces the F/A-18C cold-start task that serves as the training scenario for the system. Section 2.2 describes the visual fact ontology, a structured intermediate representation that bridges raw cockpit screenshots and downstream procedural reasoning. The remainder of the chapter positions the present work against existing research in intelligent tutoring systems, simulator-based training, state-aware tutoring, vision-language model adaptation, and grounded AI assistance.

2.1 The F/A-18C Cold-Start Task

The procedural training scenario at the centre of this thesis is the F/A-18C cold-start, a standard pre-flight cockpit procedure that brings the aircraft from a powered-down state to a taxi-ready configuration. This task is canonical in DCS World training curricula and is representative of a broader class of fixed-sequence, state-sensitive procedural tasks in which the order of actions, the verification of intermediate system states, and the correct interpretation of cockpit instrumentation all contribute to successful completion.

2.1.1 Procedure Structure: S01–S25 and Phases P1–P6

The cold-start procedure is organised into 25 numbered steps (S01–S25), grouped into six coarse-grained phases (P1–P6) that reflect distinct operational stages:

P1 — Power-up and safety (S01–S03): Battery and generator activation, fire detection system tests, and auxiliary power unit (APU) start. These steps establish electrical power and confirm critical safety systems are functional before engine ignition.

P2 — Right engine start (S04–S07): Right engine crank, throttle advance at 25% RPM, bleed air system cycling, and caution/warning/advisory lights testing. The

right engine is started first to provide hydraulic and electrical power for subsequent procedures.

P3 — Displays and communications (S08–S09): Power-up of both Digital Display Indicators (DDIs), the Advanced Multipurpose Color Display (AMPCD), and the Head-Up Display (HUD); configuration of specific display pages (FCS page on left DDI, BIT root page on right DDI); and programming of COMM1 preset frequencies via the Up-Front Controller (UFC).

P4 — Left engine and core systems (S10–S14): Left engine start following verification of right-engine parameters in nominal ranges, INS alignment mode selection (ground or carrier), radar power-up, and OBOGS (On-Board Oxygen Generating System) activation.

P5 — FCS and flight controls (S15–S19): Flight Control System (FCS) reset and BIT (Built-In Test) execution, flap and trim configuration, and the "four-down" pre-flight checks covering the refuelling probe, speed brake, launch bar, and arrestor hook.

P6 — Pre-taxi instruments and final checks (S20–S25): Parking brake release, BINGO fuel setting, barometric and radar altimeter configuration, standby attitude indicator uncaging, and attitude source selection.

The procedure is strictly ordered: each step carries explicit precondition gates that depend on the completion of earlier steps and on the satisfaction of specific telemetry conditions. For example, S04 (engine crank right) requires that the APU start support has been established (S03 completion), while S10 (engine crank left) requires that right-engine parameters — RPM, temperature, fuel flow, nozzle position, and oil pressure — all fall within their nominal green ranges before the left engine is engaged. These inter-step dependencies make the cold-start task a compelling test case for context-aware tutoring: a learner who skips a verification or misjudges an instrument reading can propagate errors through the remainder of the sequence.

2.1.2 Cockpit Displays and the Composite Panel

The visual fact extraction subsystem (ğ2.2) operates on a fixed composite panel image that captures three cockpit display regions, arranged from top to bottom:

- **Left DDI (Digital Display Indicator):** A multi-function display used for the FCS page, SUPT (Setup) page, TAC (Tactical) page, and other system pages. During the cold-start, the left DDI is primarily used for FCS monitoring (S08, S15).
- **AMPCD (Advanced Multipurpose Color Display):** The central colour display, used for the HSI (Horizontal Situation Indicator) page, INS alignment status, map layers, and navigation data. In the cold-start, the AMPCD becomes critical during the INS alignment phase (S12).
- **Right DDI:** A second multi-function display, used during the cold-start for the BIT root page (S08) and later for the FCS-MC BIT sequence (S18). The right DDI is also where BIT failure indications and FCS BIT intermediate and final results appear.

The composite panel image consolidates these three regions into a single input for the vision-language model (VLM). This design choice avoids mismatches between training and inference (where the model would be trained on per-region crops but evaluated on a full composite, or vice versa) and simplifies the multi-turn visual pipeline to a single-image extraction step per observation window [1].

2.1.3 DCS-BIOS Telemetry

DCS-BIOS provides a real-time UDP-based telemetry stream from DCS World, exporting the state of cockpit switches, indicators, warning lights, gauges, and other instruments as structured data. The SimTutor runtime receives these raw BIOS frames and passes them through a telemetry enrichment pipeline that normalises the raw values into stable runtime variables (e.g., `rpm_r`, `apu_ready`, `ins_mode`), applies delta sanitisation to suppress noise and jitter, and latches discrete state-completion flags for gate evaluation.

This telemetry feed is the primary grounding signal for the tutoring system. Unlike vision-only approaches, which must infer procedure state from pixel-level evidence alone, the SimTutor runtime can cross-reference visual observations against deterministic telemetry state. Steps such as S01 (battery and generators), S05 (right throttle idle), and S13 (radar mode OPR) are directly observable via DCS-BIOS and require no visual inference. Other steps, such as S08 and S18, involve display-page state that is not directly exported via DCS-BIOS and therefore rely on the visual fact extraction pipeline

described below.

2.2 Visual Fact Ontology

A central design decision in the SimTutor system is the separation between visual perception and procedural reasoning. Rather than training a model to directly produce a tutor decision ("the learner should now press PB5"), the system uses an intermediate structured representation called *visual facts*. A visual fact is a discrete, auditable proposition about the visible state of one cockpit display region at a single point in time. Each fact is labelled with one of three states:

`seen`: The visual evidence described by the fact is clearly visible in the current cockpit image.

`not_seen`: The visual evidence is absent or not discernible.

`uncertain`: The image is blurry, occluded, incomplete, or otherwise insufficient for a confident determination.

These facts are consumed downstream by the procedure engine (ğ**??**), which combines them with telemetry state, gating rules, and knowledge retrieval to decide whether a procedure step is visually confirmed, still pending, or blocked. This layered architecture isolates the VLM's responsibility to structured visual evidence extraction, reserving procedure-level decisions for deterministic rule-based components.

2.2.1 From 8-Fact v1 to 13-Fact v2

The visual fact ontology evolved across two generations, reflecting lessons from early fine-tuning experiments and the needs of the runtime procedure pack.

8-fact v1 (early baseline). The first-generation ontology contained eight facts targeting the three display regions. These facts covered FCS page visibility (left DDI), BIT page and BIT failure visibility (right DDI), FCS-MC page and in-test status (right DDI), FCS BIT result visibility (right DDI), INS alignment page visibility (AMPCD), and the INS GO completion signal (AMPCD). This ontology was used to train the first LoRA adapter (LoRA-v1) on 180 human-reviewed images from Run-001, yielding promising in-distribution results but exposing a critical weakness: the `ins_go` fact mixed abstraction

levels by asking the VLM to recognise a high-level procedural completion state ("INS alignment is done") from a single visual frame. The result was a high false-positive rate on the independent Run-002 holdout set (13 of 50 samples predicted `seen` when the human label was `not_seen`).

13-fact v2 (runtime-aligned). The second-generation ontology addresses this by decomposing higher-level procedure states into lower-level, visually observable evidence. The current 13-fact set is defined in `vision_facts.yaml` and includes:

- **Left DDI facts:** `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `fcs_page_x_marks_visible`.
- **Right DDI facts:** `bit_root_page_visible`, `fcsmc_page_visible`, `fcsmc_intermediate_resu` `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`.
- **AMPCD facts:** `hsi_page_visible`, `hsi_map_layer_visible`, `ins_grnd_alignment_text_vi` `ins_ok_text_visible`.

Note that `ins_go` from v1 has been decomposed into several lower-level AMPCD facts (`ins_grnd_alignment_text_visible`, `ins_ok_text_visible`, `hsi_map_layer_visible`) that each describe a specific, locally verifiable piece of visual evidence. The downstream procedure engine, rather than the VLM, is responsible for combining these atomic observations into a procedural judgement about whether the INS alignment phase is complete.

2.2.2 Sticky Facts and Step Bindings

Two additional mechanisms in the ontology improve its robustness for runtime use.

Sticky facts. Some visual facts represent transient states that, once observed, should not be overwritten by subsequent `not_seen` frames. For example, the `fcsmc_final_go_result_visibl` fact is marked as `sticky: true` with a time-to-live of 600 seconds. Once the VLM reports this fact as `seen`, it is latched in the runtime fact state and will not be overwritten by later `not_seen` or `uncertain` predictions within the TTL window. This prevents flickering and false negatives caused by display page changes, camera occlusions, or brief rendering artifacts that occur after the critical visual event has already been captured.

Non-sticky facts (e.g., `fcs_page_visible`, `bit_root_page_visible`) carry a shorter TTL of 2 seconds and are continuously re-evaluated, reflecting the expectation that these display pages may change as the learner navigates between different cockpit pages.

Step bindings. The vision facts are linked to procedure steps through explicit step bindings defined in `vision_facts.yaml`. A step binding specifies which facts must be `seen` for the procedure engine to consider the step visually confirmed. The current ontology defines two binding types:

- `all_of`: Every fact in the binding set must be `seen`. For example, S08 requires both `fcs_page_visible` (left DDI) and `bit_root_page_visible` (right DDI) to be confirmed.
- `any_of` (one-of fact collection): At least one fact in the set must be `seen`. This accommodates cases where a procedure state can be verified through any of several alternative visual cues.

Steps that rely on visual confirmation (S08, S15, S18) are registered as `vision_priority_steps` in the procedure pack, while steps directly observable through DCS-BIOS telemetry (S01, S03–S07, S09–S13, S21) are registered as `bios_priority_steps`. Steps that require manual confirmation or involve cockpit areas outside the three-display layout (S02, S14, S16–S20, S22–S25) are marked as `manual_or_out_of_layout_steps`. This tripartite classification ensures that each step's completion evidence is sourced from the most reliable available signal.

2.3 Intelligent Tutoring Systems and Procedural Training

Intelligent Tutoring Systems (ITS) have a long history in procedural training domains, from military equipment operation to medical procedure instruction. Classic ITS architectures typically model expert knowledge, track learner state through a student model, and deliver context-sensitive feedback through a tutoring module. TODO:CITE — foundational ITS survey or architecture paper (e.g., the classic four-component ITS model or a recent comprehensive survey).

In the procedural domain, constraint-based tutors and example-tracing tutors have been applied to tasks such as aircraft maintenance, programming, and equipment troubleshooting. TODO:CITE — constraint-based modeling for procedural training; TODO:CITE — example-tracing tutors or cognitive tutors in procedural domains. These systems typically operate within closed, well-defined task environments where the space of correct and incorrect actions can be enumerated. The cold-start task studied in this thesis

shares this property of a closed procedure, but differs in two key respects. First, the system observes learner behaviour through simulator telemetry and cockpit visuals rather than through a custom tutor interface, making it applicable to unmodified simulation environments. Second, the tutoring output is generated dynamically by an LLM rather than retrieved from a pre-authored library of feedback messages, allowing the system to respond to arbitrary learner states within the procedure rather than only to anticipated error patterns.

More recent work has explored the use of large language models for tutoring and instructional dialogue. TODO:CITE — LLM-based tutoring systems or dialogue-based tutors (e.g., Khanmigo-style systems, or any LLM-as-tutor paper). These systems demonstrate that LLMs can produce fluent, contextually appropriate instructional text, but they typically operate in unstructured, conversational settings and do not enforce the strict evidence-grounding constraints that characterise the SimTutor approach. In particular, the present work constrains every overlay target and step recommendation to have a verifiable evidence source — telemetry, a gating rule, a retrieved document snippet, or a visual fact — rather than relying solely on the model's parametric knowledge.

2.4 Simulator-Based Training and Instrumentation

High-fidelity simulation environments such as DCS World provide realistic, repeatable platforms for procedural training. The use of simulators in aviation training is well established, with extensive research on transfer of training from simulated to real aircraft. TODO:CITE — simulator-based training effectiveness in aviation (e.g., transfer-of-training studies, or surveys of flight simulation for training).

A key challenge in simulator-based training research is instrumentation: the means by which a training system observes learner actions and cockpit state. Prior work has instrumented simulators through screen capture, event logging, or direct state export APIs. TODO:CITE — simulator instrumentation approaches, e.g., X-Plane data output, Microsoft Flight Simulator SimConnect, or DCS-BIOS academic usage. The present work uses DCS-BIOS, a community-developed telemetry bridge that exports DCS World cockpit state via UDP. DCS-BIOS provides a richer signal than screen capture alone, exposing the exact position of switches, the numerical values of engine parameters, and the state of warning lights without requiring computer vision to recover them. This allows the system to reserve visual perception for display-page states (which are not exported by DCS-BIOS) while relying on deterministic telemetry for switch positions

and gauge readings.

TODO:CITE — any prior academic work that uses DCS World or DCS-BIOS as a research platform for AI, HCI, or training research.

2.5 Rule-Based and State-Aware Tutoring

The SimTutor system makes extensive use of explicit gating rules, deterministic step inference, and telemetry-driven state tracking alongside its LLM-based help generation. This hybrid design places it at the intersection of rule-based and neural approaches to intelligent assistance.

Rule-based approaches to procedure tracking — using finite-state machines, petri nets, or production rules to model task progress — offer strong guarantees of interpretability and correctness. TODO:CITE — rule-based procedure tracking or plan recognition for training (e.g., plan recognition for procedural tasks, or finite-state models of procedure execution). These approaches, however, typically require manual authoring of procedure models and do not generalise to novel learner errors or unexpected state configurations. The SimTutor system uses manually authored procedure packs (defining steps, gates, and evidence requirements) as the backbone of its state tracking, but delegates the natural-language explanation and overlay target selection to the LLM. This division of labour preserves the interpretability of the procedure model while enabling the flexibility of language-model-generated feedback.

Hybrid neuro-symbolic architectures have been proposed for task guidance, combining neural perception or generation with symbolic task models. TODO:CITE — neuro-symbolic task assistance or hybrid planning+LLM systems (e.g., SayCan-style grounding, or LLM+PDDL integration). The present work differs from these efforts in its emphasis on evidence grounding: every actionable output of the system must be traceable to a specific, auditable evidence source (a telemetry variable, a visual fact observation, a gating rule evaluation, or a retrieved document chunk). This constraint is not merely an architectural preference but a safety requirement: in the context of cockpit procedures, an incorrectly highlighted control or a misidentified procedure state could confuse or mislead the learner.

2.6 Foundation Models and VLMs for Situated Assistance

The adaptation of vision-language models (VLMs) to domain-specific structured extraction tasks is a rapidly growing area. Parameter-efficient fine-tuning methods, particularly Low-Rank Adaptation (LoRA), have made it feasible to adapt large pre-trained models to narrow domains with modest computational resources and small curated datasets. TODO:CITE — LoRA (Hu et al., 2021); TODO:CITE — QLoRA or other PEFT methods used for VLM fine-tuning; TODO:CITE — representative work on fine-tuning VLMs for domain-specific visual question answering or structured output tasks.

The VLM adaptation pipeline in this thesis (ğ**??**) follows a capture-prelabel-review-train-benchmark loop that is notable for its use of human-reviewed small data (hundreds of images, not tens of thousands) to drive meaningful improvements in structured extraction accuracy. This contrasts with large-scale VLM fine-tuning efforts that rely on web-scale data and broad instruction tuning. TODO:CITE — work on small-data domain adaptation of VLMs or LLMs, few-shot task-specific fine-tuning.

Prior work on visual understanding of graphical user interfaces and structured screen content has explored tasks such as widget detection, screen summarisation, and UI state classification. TODO:CITE — UI understanding or screen parsing with VLMs (e.g., Screen2Words, Widget Captioning, or vision-based UI automation). The cockpit display domain studied here shares structural similarities with GUI understanding — fixed regions, structured layouts, symbolic content — but differs in the criticality of the task and the need for explicit, auditable evidence rather than open-ended descriptions.

The bilingual SFT strategy used in this work (training both English and Chinese variants from the same images and labels) is related to cross-lingual transfer and instruction diversity in fine-tuning. TODO:CITE — cross-lingual instruction tuning or multilingual SFT for VLMs. However, the primary motivation here is not multilingual deployment but rather instruction diversity as a form of regularisation: by training on two languages, the model is encouraged to attend to the visual content rather than to language-specific priors, while the structured JSON output format remains language-independent.

2.7 Explainable and Grounded AI Assistance in AR/VR

The original thesis proposal envisioned an in-headset augmented reality / virtual reality tutoring interface running on the Meta Quest 3. While the current implementation

operates on a desktop DCS runtime (see ğ1.3), the long-term research agenda remains anchored in immersive procedural training.

Research on augmented reality guidance for procedural tasks has demonstrated the value of spatially registered instructions for assembly, maintenance, and repair tasks. TODO:CITE — AR-guided procedural training survey (e.g., AR for assembly, maintenance, or medical procedure guidance). These systems typically use pre-authored content (arrows, text, animations) registered to known physical objects. The SimTutor vision differs in that the instructional content is generated at runtime from the current procedure state and visual evidence, rather than retrieved from a fixed library.

Work on explainable AI in decision-support systems has emphasised the importance of traceable reasoning, particularly in safety-critical domains. TODO:CITE — explainable AI for decision support or safety-critical AI assistance (e.g., XAI in aviation, or explainable recommenders for high-stakes tasks). The evidence-grounding constraint in SimTutor aligns with this principle: each overlay target and recommended step must carry an evidence reference that links the output to its source (a telemetry variable, a gating rule, a visual fact, or a retrieved document chunk). This audit trail is designed to support both runtime debugging (why did the system highlight this control?) and post-hoc analysis of help cycles.

TODO:CITE — any work on LLM-based or AI-driven assistance in VR/AR that addresses grounding, evidence, or procedural correctness (as opposed to open-ended conversation).

2.8 Positioning of This Work

The SimTutor system occupies a position at the intersection of several research areas that are rarely combined in a single system: it applies a **telemetry-grounded** approach rather than a vision-only or text-only approach to procedural state tracking; it uses **parameter-efficient VLM adaptation** to produce structured visual facts rather than free-form descriptions; it enforces **explicit evidence grounding** on all actionable outputs; and it treats the **procedure pack** as a first-class configuration artifact that can be authored, versioned, and replayed independently of the model.

From the perspective of intelligent tutoring systems, SimTutor contributes a design in which the tutoring logic is distributed across three layers: a deterministic pack-driven procedure engine for state tracking, an LLM for grounded help generation, and a fine-tuned VLM for display-page visual fact extraction. From the perspective of VLM

adaptation research, it contributes a domain-specific benchmark and ontology evolution narrative (8-fact to 13-fact) that makes explicit the gap between early visual targets and the granularity required for reliable procedure-level reasoning. From the perspective of simulator-based training, it contributes a reusable instrumentation and replay infrastructure that can support future human-subject studies without requiring real-time experimenter oversight.

The work presented in this thesis represents a technical baseline rather than a completed evaluation. The VLM adaptation results (ğ6) provide evidence that structured visual fact extraction can be substantially improved through small-data domain fine-tuning, but they do not yet constitute evidence of learning gains, workload reduction, or transfer. The human-subject evaluation envisioned in the original thesis proposal remains a planned next phase (ğ6.2), and the current system is best understood as the runtime and instrumentation infrastructure that makes that evaluation feasible.

3 System Design

This chapter describes the design of SimTutor, a telemetry-grounded tutoring runtime for procedural training in the DCS F/A-18C cold-start task. It covers the system architecture, component design, and design rationale. Detailed methodology—including the VLM adaptation protocol, dataset curation, and benchmark design—is deferred to Chapter 4; implementation-level details of specific modules appear in Chapter 5. Section 3.1 derives the design goals from the thesis problem statement. Section 3.2 presents the overall architecture and the separation of core tutoring logic from simulator-specific adapters. Section 3.4 describes the runtime data flow through the main help cycle. Sections 3.5 and 3.6 cover the telemetry processing pipeline and the procedure and gating rule engines. Section 3.7 details the simulator adapter layer and Lua-side integration. Section 3.8 describes the event logging, replay, and scoring infrastructure. Section 3.9 presents the VLM adaptation component for visual state understanding. Section 3.10 concludes with a systematic comparison to the original thesis proposal, documenting how the design evolved during implementation.

3.1 Design Goals and Requirements

The thesis addresses the problem of providing grounded, context-aware procedural guidance to a learner operating a complex simulator. In the target scenario—the F/A-18C cold-start procedure in DCS World—the learner must execute a fixed sequence of 25 steps (S01–S25) spanning six phases (P1–P6), from battery power-up through engine start, avionics configuration, flight-control checks, and taxi preparation. Each step has specific preconditions that must be satisfied before execution and completion conditions that must be met before advancing. Mistakes include omissions, order errors, parameter-setting errors, and state violations—categories formalised in the error taxonomy described in Chapter 4.

From this problem, we derive the following design goals:

G1 – Simulator-based procedural tutoring. The system must observe the learner's ac-

tions through simulator state, determine which procedural step the learner is currently attempting, and provide timely, grounded guidance when the learner is stuck or requests help. The tutoring output must reference concrete cockpit elements and cite evidence from telemetry, documentation, or visual facts.

G2 – State-aware step guidance. The system must track procedure progress through a formal state machine. It must evaluate preconditions before allowing a step to become active and verify completion conditions before advancing. Guidance must be conditional on the current simulator state, not merely on step order.

G3 – Separation between tutoring logic and simulator-specific integration. The core tutoring logic—procedure tracking, gating, scoring, and help generation—must be independent of any particular simulator, model provider, or knowledge source. Simulator-specific code, model-provider code, and knowledge-retrieval code must be isolated behind stable interfaces so that the core can be tested without a running simulator and can be ported to other simulators or training domains.

G4 – Replayability and inspectability. Every interaction between the learner, the tutoring system, and the simulator must be recorded as structured, timestamped events. These event logs must support offline replay for debugging, automated scoring against a procedure pack, and post-hoc analysis of learner behaviour. They must also support replay of telemetry captures and benchmark traces for the VLM adaptation pipeline.

G5 – Foundation-model and VLM-based visual fact extraction. The system must be able to operate with or without a large language model. When a model is available, it provides richer diagnoses and explanations. When unavailable or when its output fails validation, the system must degrade gracefully to deterministic, rule-based fallback behaviour. Vision-language model (VLM) based visual fact extraction from cockpit screenshots is a core system component that complements telemetry gating by providing structured visual evidence for display-page states not exported through DCS-BIOS.

G6 – Extensibility to different simulators and procedures. The procedure definition, UI target mapping, telemetry mapping, error taxonomy, and knowledge source policy must be expressed as declarative configuration (YAML files) rather than hard-coded in source code. Adding a new procedure or adapting the system to a

different simulator should require only new configuration artifacts, not changes to the core tutoring logic.

These goals directly inform the architectural decisions described in the following sections.

3.2 Overall Architecture

SimTutor follows a ports-and-adapters (hexagonal) architecture. The system is organised into three main layers, each with clearly defined responsibilities and dependency directions.

Core layer (`core/`). The core contains all domain logic that is independent of any particular simulator, model provider, or external service. It defines the data contracts for runtime messaging (Section 3.4), the procedure state machine and gating rule engine (Section 3.6), the scoring and interaction-metrics engines, the vision fact ontology and sticky/TTL persistence semantics, the BM25 retrieval implementation, the HelpResponse schema builder, and the security and audit infrastructure. By design, the core depends only on standard Python libraries and the abstract interfaces defined in the ports layer.

Ports layer (`ports/`). The ports layer defines stable, technology-agnostic Python protocols for the four external dependencies the core may need at runtime: model reasoning (`ModelPort`), knowledge retrieval (`KnowledgePort`), vision input (`VisionPort`), and telemetry input (`TelemetryPort`). These interfaces are the only dependency injection points between the core and the outside world.

Adapters layer (`adapters/`). The adapters layer contains all simulator-specific, provider-specific, and infrastructure-specific code. It includes DCS-BIOS telemetry receivers, the telemetry enrichment and delta-sanitisation pipeline, DCS overlay senders, OpenAI-compatible model clients (with and without multimodal support), Ollama fallback clients, a deterministic model stub for testing, a local BM25 knowledge adapter, the vision fact extraction runtime, and the response-mapping and action-execution components that bridge model output to runtime behaviour. Adapters depend on the ports they implement and on the core types they produce and consume; they never introduce reverse dependencies into the core.

The three layers are complemented by two further structural elements:

Procedure packs (`packs/`). A procedure pack is a directory of declarative YAML configuration files that fully describe one procedural task. The pack defines the step list, preconditions and completion conditions (gating rules), UI targets and their simulator-specific element identifiers, telemetry variable mappings, error taxonomy categories and scoring weights, delta-sanitisation policy, vision fact ontology with step bindings, and vision layout description. The current repository contains one pack: `fa18c_startup`, covering the full F/A-18C cold-start procedure.

CLI and runtime entrypoints (`simtutor/`, `live_dcs.py`). The `simtutor` package provides a command-line interface for running mock scenarios, replaying and validating event logs, scoring runs, recording VLM predictions for benchmarking, and replaying DCS-BIOS raw captures. The `live_dcs.py` module orchestrates the full live runtime loop: it wires together a DCS-BIOS receiver, the telemetry enrichment pipeline, knowledge retrieval, model-based help generation, VLM visual fact extraction, overlay action execution, and event logging into a single long-running process.

Figure 3.1 shows the high-level component diagram.

Figure 3.1: SimTutor system architecture. The core layer contains simulator-agnostic domain logic. The ports layer defines abstract interfaces. The adapters layer provides concrete implementations for DCS, model providers, knowledge sources, and the vision pipeline. Procedure packs supply declarative configuration.

3.3 Separation of Core Tutoring Logic and Simulator Adapters

The separation between simulator-independent core logic and simulator-specific adapters serves four concrete purposes:

1. **Testability.** The core logic can be tested with mock adapters and deterministic scenarios without requiring a running DCS instance, a GPU, or network access to a model server. The repository includes a `MockEnvAdapter` that replays pre-recorded observation sequences and a `ModelStub` that returns deterministic responses, enabling fast, reproducible unit and integration tests.

2. **Maintainability.** When a simulator API changes, only the affected adapter needs to be updated; the core remains unchanged. When a model provider changes (e.g., from Ollama to an OpenAI-compatible vLLM server), only the model adapter configuration is affected. Three providers (`openai_compat`, `ollama`, `stub`) are supported through a single configuration interface.

3. **Transferability.** Porting the system to a new simulator or aircraft requires writing new telemetry and overlay adapters and creating new procedure packs. The procedural reasoning, gating, scoring, and help-generation infrastructure can be reused as-is. The port interfaces are intentionally minimal: a telemetry port requires `start`, `poll`, and `stop` methods; an overlay port requires only the ability to send highlight, clear, and pulse commands to named cockpit elements.

4. **Independent evolution of the VLM pipeline.** The VLM visual fact extraction component communicates with the tutoring runtime through the vision port and the vision fact observation schema, not through direct coupling. The VLM pipeline can be developed, trained, and benchmarked independently of the tutoring runtime. A runtime deployment can include the VLM component, omit it entirely, or replace it with a different perception backend without affecting the tutoring logic.

3.4 Data Flow

The runtime data flow follows a structured pipeline from simulator observation to tutoring output and logging. Figure 3.2 provides a visual summary.

Figure 3.2: Main runtime data flow through the help cycle.

3.4.1 Observation

The simulator adapter—in the current implementation, a DCS-BIOS UDP receiver—produces a stream of *observations*: versioned, timestamped data objects containing raw simulator state and, after enrichment, a set of normalised state variables.

3.4.2 Tutor Request

When the learner triggers help—via a hotkey or a UDP message—a *tutor request* is created. The request carries an intent (`help`), an optional free-text message, a reference to the current observation, and a context dictionary populated by the runtime orchestrator.

3.4.3 Context Assembly

The runtime orchestrator assembles a context for the help cycle by gathering:

- **State variables** (`vars`): normalised key–value pairs from the most recent telemetry observation.
- **Recent deltas** (`recent_deltas`): a sanitised, aggregated summary of recent state changes.
- **Candidate steps** (`candidate_steps`): eligible step identifiers from the procedure pack.
- **Deterministic step hint** (`deterministic_step_hint`): a locally computed fallback suggestion from rule-based inference, used both as prompt guidance and as fallback when model output is unavailable.
- **Overlay target allowlist** (`overlay_target_allowlist`): the set of UI elements the model is permitted to reference.
- **Knowledge snippets** (`rag_topk`, optional): top-k BM25 retrieval results, included only when a knowledge index exists.

3.4.4 Model Inference

If a ModelPort implementation is available, the assembled context is formatted into a prompt and sent to the language model. The model returns a structured JSON object conforming to the HelpResponse schema, which is generated dynamically at runtime from the procedure pack, UI map, and taxonomy.

3.4.5 Response Validation and Mapping

The raw model output passes through a validation pipeline: (1) JSON extraction with minimal repair, (2) schema validation, (3) allowlist checks on diagnosis step IDs, next

step IDs, and overlay targets, (4) evidence-grounding validation. If validation succeeds, the help object is mapped to a `TutorResponse` with overlay actions resolved through the UI map. If validation fails, or if no model is available, the system enters deterministic fallback.

3.4.6 Action Execution

The tutor response may contain overlay actions (highlight, clear, or pulse) targeting specific cockpit elements. The action executor validates each action against the UI target allowlist, enforces a limit on concurrent highlights (default: one), and dispatches the command to the simulator via UDP. Auto-click and control-injection actions are forbidden by design.

3.4.7 Event Logging

Every stage of the help cycle is recorded as a versioned, timestamped `Event` object and appended to a JSONL event log, which serves as the authoritative record for all downstream analysis, replay, and scoring.

3.5 Telemetry Processing

The telemetry processing subsystem transforms raw simulator data into the normalised state representation through four stages:

1. **Raw ingestion.** The DCS-BIOS receiver listens on a UDP socket for binary DCS-BIOS export frames, decoding each into a structured JSON object with wall-clock timestamps and monotonically increasing sequence numbers.

2. **Variable resolution.** A telemetry map specifies how raw BIOS fields are projected onto normalised state variables. The variable resolver produces a flat dictionary of key–value pairs and tracks unavailable or unknown sources in a `vars_source_missing` set, enabling downstream components to distinguish "the value is zero" from "the value is unknown."

3. **Delta sanitisation.** A configurable policy (thresholds, debounce windows, per-variable filtering rules) suppresses spurious changes caused by switch bounce, sensor drift, and UDP out-of-order delivery.

4. **Delta aggregation.** Sanitised deltas are accumulated over a configurable time window and compressed into a compact summary for the prompt context.

The telemetry pipeline runs continuously, independent of the help cycle.

3.6 Procedure Engine and Gating Rule Engine

3.6.1 Procedure Engine

The procedure engine maintains a finite-state machine over the ordered sequence of steps. Each step is in one of four states: `pending`, `active` (at most one), `done`, or `blocked`. Operations include `activate_next`, `complete_active`, `block_active`, `rewind`, and `check_timeout`. All transitions are recorded as internal events and propagated to the event log.

3.6.2 Gating Rule Engine

The gating rule engine evaluates precondition gates and completion gates using a v1 DSL with four operators: `var_gte`, `arg_in_range`, `flag_true`, and `time_since`. The evaluator includes explicit unknown-value handling: missing, source-missing, or sentinel-value variables cause rule failure with a diagnostic reason rather than silent treatment as false. Rules short-circuit on first failure; the failure index and reason feed into the deterministic step hint.

3.6.3 Deterministic Step Inference

When no model is available or model output fails validation, the system falls back to deterministic step inference, which examines the active step, unsatisfied gating rules, and recent UI targets to produce a text hint restricted to the currently active step. By default, no overlay highlights are sent in fallback mode unless a local rule can prove a unique, valid target given the current telemetry state.

3.6.4 Procedure Pack Structure

The procedure pack is the declarative configuration driving the entire tutoring runtime. Each step is classified by primary evidence source:

- **BIOS-priority steps** (S01, S03–S07, S09–S13, S21): completion evidence primarily in telemetry variables.
- **Vision-priority steps** (S08, S15, S18): require visual confirmation through cockpit displays.
- **Manual or out-of-layout steps** (S02, S14, S16, S17, S19, S20, S22–S25): cannot be fully observed through either telemetry or the current visual layout.

The pack also defines per-step gating rules, UI targets, tutor prompts, and profile overrides (airfield vs. carrier).

3.7 Simulator Adapter Layer

3.7.1 Telemetry Input Adapter

Receives DCS-BIOS state data over UDP. The Python-side `DcsBiosRawReceiver` parses the binary protocol and emits raw frames as JSONL records, which feed the telemetry enrichment pipeline.

3.7.2 Overlay Output Adapter

Sends highlight, clear, and pulse commands to DCS via plain-text UDP messages. A Lua script (`VRHilite.lua`) inside DCS renders graphical overlays on cockpit elements. The adapter maps abstract target names to DCS-internal point codes using the procedure pack's UI map.

3.7.3 Vision Input Adapter

Triggers screenshot captures in DCS via a UDP-based capture trigger. The Lua-side `SimTutor.lua` renders the three primary cockpit displays into a single composite image. The Python-side adapter selects pre-trigger and trigger frames and passes them to the vision fact extractor.

3.7.4 Lua-Side Integration

Three categories of Lua scripts execute inside DCS:

- **State export** (`Export.lua`): configures DCS-BIOS export.

- **Overlay rendering** (`Hooks/VRHilite.lua`, `Hooks/SimTutorHighlight.lua`): receives and renders highlight commands.
- **SimTutor integration** (`SimTutor/SimTutor.lua`, `SimTutor/SimTutor Function.lua`, `SimTutor/SimTutorDcsBiosHub.lua`): handles capture and composite-panel rendering.

3.7.5 Planned and Partially Implemented Adapters

The current repository includes a `MockEnvAdapter` for deterministic testing without a running simulator. The VR runtime code path is implemented, including VR_MIRROR and VR_allow_MFD_out_of_HMD configuration for Quest 3 VR mode, tutor text UDP injection into the DCS in-game text area, and composite-panel capture compatibility validation in VR mode. The VR code path has not yet been validated through controlled human-subject experiments (Section 6.2).

3.8 Event Logging and Replay

3.8.1 Event Model

All runtime occurrences are represented as `Event` objects with a fixed schema: unique identifier, wall-clock timestamp, session identifier, event kind (`observation`, `tutor_request`, `tutor_response`, `overlay_requested`, `overlay_applied`, `overlay_failed`), typed payload, schema version, and extensible metadata.

3.8.2 JSONL Event Store

Events are persisted in JSONL format via `JsonlEventStore`, supporting append-only writing and full-file loading.

3.8.3 Replay and Validation

A recorded event log can be replayed offline to verify step ordering and state machine consistency. Telemetry logs are independently validated for monotonic sequence numbers and timestamps. The replay infrastructure has been implemented at the code level and exercised with recorded telemetry traces; it has not yet been validated with end-to-end human-in-the-loop sessions.

3.8.4 Scoring and Interaction Metrics

The scoring engine counts omissions and state violations with configurable per-category weights and a critical-step multiplier. Interaction metrics characterise learner behaviour: stall time, help requests, LLM triggers, UI click counts, and overlay request/acknowledgement counts.

3.8.5 Current Limitations

The replay infrastructure currently supports synthetic regression replay at the telemetry level, including scenarios with vision frame sidecars (`replay_eval/fa18c_startup_v04/`). End-to-end replay of full multimodal sessions with synchronised vision frames from live human runs has not yet been validated.

3.9 VLM Adaptation for Visual State Understanding

Visual perception in SimTutor is a core, independently operable subsystem.

3.9.1 Role of the VLM Component

The VLM component extracts structured *visual facts* from cockpit screenshots—intermediate propositions about visible display content. Visual facts are not final tutoring decisions; they provide visual evidence that the tutoring runtime combines with telemetry, procedure state, and documentation. This decomposition enables each component to be developed, tested, and evaluated independently.

3.9.2 Vision Fact Ontology

The current ontology (v2) defines 13 facts spanning three display regions:

- **Left DDI**: `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `fcs_page_x_marks_vi`
- **Right DDI**: `bit_root_page_visible`, `fcsmc_page_visible`, `fcsmc_intermediate_result_vi` `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`.
- **AMPCD**: `hsi_page_visible`, `hsi_map_layer_visible`, `ins_grnd_alignment_text_visible`, `ins_ok_text_visible`.

Each fact has a three-valued state (`seen`, `not_seen`, `uncertain`), a time-to-live, an optional `sticky` flag, and step bindings via `all_of` / `any_of` conditions.

3.9.3 VLM Integration in the Help Cycle

When a help request arrives and the active step is vision-priority (S08, S15, S18), a capture trigger is sent to DCS. Pre-trigger and trigger frames are rendered and sent to the VLM. The returned facts are merged into the vision fact snapshot (with sticky and TTL semantics) and included in the help-cycle context. If the vision adapter is unavailable or the multimodal request fails, the help cycle proceeds with telemetry and knowledge only.

3.9.4 VLM Adaptation Pipeline

The VLM used at runtime is fine-tuned. The adaptation pipeline—detailed in Chapter 4—comprises screenshot capture, pre-labeling by a large VLM, human review in Label Studio, bilingual SFT export, LoRA fine-tuning (Unsloth + PEFT + TRL), and base-vs-LoRA benchmarking on independent held-out sessions. This pipeline forms a self-contained contribution spanning data capture through benchmark evaluation.

3.9.5 Distinction from Rule-Based State Gating

Rule-based gating evaluates telemetry variables deterministically; it is deterministic and auditable but limited to telemetry-representable conditions. VLM-based visual fact extraction interprets display content that has no telemetry representation. The two mechanisms are complementary: the gating engine provides the primary procedure-tracking backbone; visual facts supply supplementary evidence for steps whose completion cannot be inferred from telemetry alone.

3.10 Differences from the Original Proposal

The system described in this chapter differs in several important respects from the system envisioned in the original thesis proposal. We document these differences to establish a clear boundary between the original research vision and the current implemented system.

3.10.1 From VR-First to Dual-Platform Runtime

The original proposal described a Meta Quest 3 VR tutoring system with in-headset step cards. The current implementation supports both desktop/DCS and VR (Quest 3) configurations, with cockpit overlay highlighting as the primary output modality. The

VR runtime code path is implemented (VR_MIRROR configuration, tutor text UDP injection, composite-panel capture in VR mode) and the desktop configuration provides a controlled environment for systematic benchmarking. The gap between proposal and current work is that the VR code path has not yet been validated through controlled human-subject experiments (Section 6.2).

3.10.2 Added VLM Adaptation Pipeline

The original proposal envisioned a text-only RAG+LLM tutor. The VLM visual fact extraction pipeline was added after discovering that several critical procedure steps require visual confirmation not inferable from telemetry alone. The VLM pipeline evolved into an adaptation and benchmarking loop that now constitutes a substantial technical contribution.

3.10.3 Expanded Telemetry Grounding

The current implementation includes a substantially more elaborate telemetry subsystem: raw DCS-BIOS UDP ingestion, variable resolution with source-missing tracking, delta sanitisation, and delta aggregation. This expansion was driven by the practical realities of working with noisy, high-frequency simulator telemetry.

3.10.4 Added Procedural, Replay, and Benchmark Infrastructure

The original proposal did not describe a formal procedure engine, gating rule engine, error taxonomy, event logging, replay, or automated benchmarking. These were developed during implementation as essential infrastructure for systematic evaluation.

3.10.5 Postponed Human-Subject Evaluation

The original proposal planned a three-condition human-subject study that has not been conducted. The current repository contains the technical infrastructure to support such a study but no participant data. The human-subject evaluation remains the central planned component (Section 6.2).

3.10.6 Summary of Design Evolution

Table 3.1 summarises the key differences.

Table 3.1: Design evolution from proposal to implemented system.

Aspect	**Original Proposal**	**Implemented System**
Platform	Meta Quest 3 VR	Desktop / DCS and VR (Quest 3)
Output modality	In-headset step cards	Cockpit overlay highlights
State input	Read-only DCS state flags	Full DCS-BIOS telemetry with delta sanitisation
Visual perception	None (text-only RAG)	VLM visual fact extraction (13-fact v2)
Model adaptation	Not addressed	LoRA fine-tuning pipeline (primary line: Qwen3.5-9B; supplementary: Gemma-4-31B)[1]
Procedure representation	Implicit in prompts	Explicit pack-driven configuration with gating rules
Error handling	Not specified	Structured taxonomy, deterministic fallback, and action restrictions
Evaluation infrastructure	Not specified	Event logging, replay validation, automated scoring, benchmark pipeline
Human-subject study	Planned as main evaluation	Postponed; infrastructure prepared

This evolution does not invalidate the original proposal. Rather, the implemented system provides a concrete technical foundation—with running code, automated tests, and benchmark artifacts—on which the planned human-subject evaluation can be built.

[1] Qwen3.5-9B (Qwen/Qwen3.5-9B-Base) fine-tuning and benchmark results are the primary verified experimental line (C-VLM-04, C-VLM-05). Gemma-4-31B (google/gemma-4-31B) results are supplementary (C-VLM-06, *likely*).

4 Methodology

This chapter describes the methodological framework underlying the implemented SimTutor system. Section 4.1 covers the telemetry processing pipeline that transforms raw DCS-BIOS data into normalised runtime state. Section 4.2 describes the knowledge grounding subsystem, including BM25 retrieval and source policy enforcement. Section 4.3 presents the seven-stage visual fact data pipeline from screenshot capture through LoRA fine-tuning to benchmark evaluation. Section 4.4 details the six curated datasets, the ontology evolution from 8-fact v1 to 13-fact v2, and the training-recipe construction. Section 4.5 specifies the LoRA fine-tuning configuration, including exact hyperparameters for both backbone models. Section 4.6 defines the benchmark protocol, metrics, and holdout strategy. The chapter provides the methodological foundation for the system design presented in Chapter 3 and the technical results reported in Chapter 6.

4.1 Procedure Runtime and Telemetry Grounding

The procedure runtime is the central mechanism by which SimTutor tracks learner progress through the F/A-18C cold-start procedure and produces grounded, state-aware tutoring output. It ingests raw simulator telemetry, normalises it into stable state variables, evaluates gating rules to determine step eligibility, and provides the resulting state representation to the help-cycle context assembler. This section describes each stage of the telemetry processing chain and the gating rule evaluation framework.

4.1.1 DCS-BIOS Raw Frame Ingestion

The telemetry subsystem begins with raw DCS-BIOS data ingestion. The DCS-BIOS receiver listens on a UDP socket for binary DCS-BIOS export frames transmitted by the simulator. Each frame is decoded into a structured JSON object with wall-clock timestamps and monotonically increasing sequence numbers. The decoded frames are written to a JSONL file for archival and replay purposes and forwarded into the telemetry enrichment pipeline.

The DCS-BIOS protocol covers a wide range of cockpit controls and indicators—switch positions, knob states, warning lights, and display-related state flags—but the raw frames are noisy. Switch bounce, sensor drift, rapidly fluctuating gauge values, and out-of-order UDP delivery introduce spurious variations that, if passed directly to the tutoring runtime, would produce false state-change events and trigger unnecessary help cycles. The subsequent pipeline stages address these issues systematically.

4.1.2 Variable Resolution and Source-Missing Tracking

A telemetry map (specified in the procedure pack's `telemetry_map.yaml`) defines how raw DCS-BIOS fields are projected onto normalised state variables. The variable resolver consumes an enriched telemetry observation and produces a flat dictionary of key–value pairs, where each key corresponds to a named variable in the procedure pack (e.g., `apu_ready`, `battery_switch_on`).

Crucially, the resolver tracks unavailable or unknown sources in a `vars_source_missing` set. This mechanism enables downstream components to distinguish "the value is zero" from "the value is unknown," which is essential for correct gating rule behaviour. A missing variable never silently evaluates to a successful condition.

4.1.3 Gating Rule Evaluation

The procedure engine evaluates two types of gates for each step: precondition gates, which must be satisfied before a step becomes active, and completion gates, which must be satisfied before the step is marked done. The gating rule DSL supports four operators:

`var_gte` — a named variable must be greater than or equal to a threshold.

`arg_in_range` — a named variable must fall within a specified range.

`flag_true` — a boolean flag must be true.

`time_since` — a minimum time must have elapsed since a specified event.

The evaluator includes explicit unknown-value handling: missing, source-missing, or sentinel-value variables cause rule failure with a diagnostic reason rather than silent treatment as false. Rules short-circuit on first failure, and the failure index and reason are propagated to downstream components, including the deterministic step hint and the help-cycle context.

Gating rules are specified declaratively in the procedure pack's configuration. Each step in the F/A-18C cold-start procedure has been assigned precondition and completion gates based on the procedure's operational requirements. Steps classified as BIOS-priority (S01, S03–S07, S09–S13, S21) rely primarily on telemetry-grounded gating rules for completion detection. Steps classified as vision-priority (S08, S15, S18) supplement gate evaluation with visual fact evidence, since their completion conditions involve cockpit-display states not reflected in telemetry.

4.1.4 Delta Sanitisation and Aggregation

Raw telemetry variables change at high frequency due to simulator physics, even when no learner action has been performed. To produce a stable state representation suitable for prompt context, the pipeline applies two successive processing stages.

Delta sanitisation. A configurable policy (specified in the procedure pack's `delta_policy.yaml`) defines per-variable thresholds, debounce windows, and filtering rules. Spurious changes caused by switch bounce, sensor noise, and UDP out-of-order delivery are suppressed. Only changes that exceed the configured thresholds and persist beyond the debounce window are promoted to the sanitised delta stream.

Delta aggregation. Sanitised deltas are accumulated over a configurable time window and compressed into a compact, human-readable summary. This summary is included in the help-cycle context as `recent_deltas`, providing the language model with a concise representation of recent learner actions and state transitions without exposing raw high-frequency telemetry.

4.1.5 Deterministic Fallback

When no language model is available or when model output fails validation, the procedure runtime falls back to deterministic step inference. The fallback mechanism examines the currently active step, its unsatisfied gating rules, and recent UI targets to produce a text hint restricted to the currently active step. By default, no overlay highlights are sent in fallback mode unless a local rule can prove a unique, valid target given the current telemetry state. This ensures that the system remains operational and safe even without a model.

4.2 Knowledge Grounding and Source Policy

The knowledge grounding subsystem provides the tutoring runtime with retrieval-augmented procedural documentation. It retrieves relevant text snippets from an indexed document collection and enforces a source policy that restricts which documents may be surfaced to the model.

4.2.1 BM25 Retrieval

Knowledge retrieval uses BM25, a bag-of-words ranking function from the probabilistic information retrieval family. Documents are indexed at the sentence level using the `index_docs.py` tool, which preprocesses the F/A-18C NATOPS flight manual and supplemental cold-start documentation into a local JSON index.

At retrieval time, a grounding query is constructed from the current procedure state: the active step identifier, its description from the step registry, and the most recent telemetry observation. The query is tokenised and scored against the indexed sentences using the standard BM25 weighting function with default parameters ($k_1 = 1.2$, $b = 0.75$). The top-k scoring snippets (default $k = 5$) are returned and included as optional context in the help-cycle prompt.

4.2.2 Source Policy

Not all knowledge sources are appropriate for every tutoring context. A declarative source policy (specified in `knowledge_source_policy.yaml`) defines an allowlist of permitted document sources. Each indexed sentence carries a source identifier, and the policy enforces that only snippets from allowed sources are included in the retrieval results.

The source policy serves two purposes. First, it prevents the model from being exposed to procedure variants that contradict the current procedure pack—for example, carrier-based procedures when the current profile is airfield. Second, it bounds the retrieval scope to curated, verified documentation, reducing the risk of hallucination from outdated or irrelevant sources.

The knowledge subsystem is optional: if no knowledge index exists, the help cycle proceeds with telemetry and procedure context only. Missing knowledge retrieval must not block the main tutoring flow.

4.3 Visual Fact Data Pipeline

The visual fact data pipeline transforms raw DCS cockpit screenshots into structured visual fact extraction models. It comprises seven stages, spanning data capture, human annotation, supervised fine-tuning dataset generation, model training, and benchmark evaluation. Each stage produces versioned, inspectable artifacts, enabling full traceability from raw image to benchmark metric. Figure 4.1 provides an overview.

Figure 4.1: Seven-stage visual fact data pipeline. Capture and prelabeling feed human review; reviewed labels drive bilingual SFT export; exported data trains a LoRA adapter; base and LoRA models are benchmarked on independent holdout sets.

4.3.1 Stage 1: DCS Screenshot Capture

The first stage captures cockpit screenshots from DCS World. The Lua-side `SimTutor.lua` script renders the three primary multifunction displays—left DDI, AMPCD, and right DDI—into a single composite-panel image. The Python-side `capture_vlm_dataset.py` script orchestrates the capture process: it triggers screenshots at configurable intervals, associates each capture with a session identifier and wall-clock timestamp, and stores the raw images alongside a sidecar manifest.

The composite-panel format is chosen to keep training, pre-labeling, evaluation, and runtime usage on the same input distribution, avoiding a train–evaluation mismatch where the model is exposed to three separate region crops during training but receives a single composite image at inference time.

4.3.2 Stage 2: Initial VLM Pre-Labeling

Each captured composite-panel image is pre-labeled by a large VLM (a Qwen 397B-class model accessed via an OpenAI-compatible API) using the target visual fact ontology as the prompt contract. The pre-labeling script sends each image alongside a structured prompt that enumerates all facts in the ontology, their definitions, and the three admissible states (`seen`, `not_seen`, `uncertain`). The VLM returns a JSON object containing per-fact labels and evidence notes.

Pre-labeling serves as a seeding step: it produces initial annotations at scale, which are then refined by human review. The pre-labeling VLM is large and general-purpose;

it is not fine-tuned on the cockpit domain and frequently makes errors on ambiguous or rare visual states. Human review in the next stage corrects these errors.

4.3.3 Stage 3: Label Studio Human Review

The pre-labeled samples are imported into Label Studio, an open-source data annotation platform. Each task presents the reviewer with the composite-panel image, the VLM's predicted labels for all facts, and a free-text summary field. The reviewer inspects the image, corrects any mislabeled facts, and provides an evidence note justifying each correction. The review interface enforces that every fact receives one of the three admissible states and that the summary field is populated.

Human review is the quality-control bottleneck: each capture session contains between 50 and 220 images, and review time per image depends on the number of visible facts and the reviewer's familiarity with cockpit displays. All reviewed samples carry an audit trail linking back to the raw image, the original VLM pre-label, and the reviewer's corrections.

4.3.4 Stage 4: Reviewed JSONL Export

After human review, Label Studio exports the corrected labels as a JSONL file. Each line contains a task identifier, the original image path, the reviewed facts with per-fact state and evidence note, and the reviewed summary. This reviewed JSONL is the authoritative ground-truth artifact for all downstream stages (SFT export, training, and benchmark).

4.3.5 Stage 5: Bilingual SFT JSONL Generation

The reviewed JSONL is transformed into an OpenAI-compatible multimodal chat SFT format by `export_vision_sft_dataset.py`. Each reviewed sample produces two training rows—one in English and one in Chinese—sharing the same composite-panel image and the same fact labels but differing in the system and user instruction texts.

Each SFT row is a three-message conversation: a system message instructing the model to output JSON only, a user message containing the base64-encoded image and the textual instruction (which enumerates all facts, their definitions, and decision boundaries), and an assistant message containing the ground-truth JSON object with the `facts` array and a `summary` string. Each fact entry in the assistant output contains exactly

three fields: `fact_id`, `state`, and `evidence_note`. External metadata fields such as `frame_id`, `session_id`, `confidence`, and image paths are excluded from the training target, since they are dataset-management fields, not visually inferable properties.

Bilingual SFT increases instruction diversity rather than visual diversity: English and Chinese examples share identical images and fact labels. The purpose is to improve JSON-format robustness under both languages and to support eventual bilingual runtime deployment.

4.3.6 Stage 6: LoRA Fine-Tuning

The exported SFT JSONL files are used to fine-tune a base VLM via Low-Rank Adaptation (LoRA). The training stack combines three components: Unsloth for 4-bit VLM loading, LoRA injection, and vision batch collation; PEFT (Parameter-Efficient Fine-Tuning) for LoRA adapter definition; and TRL `SFTTrainer` for the supervised fine-tuning loop.

The training script loads the base model in 4-bit precision via Unsloth's `FastVisionModel.from_pret` applies LoRA adapters to both vision and language layers (Qwen) or language-only linear layers (Gemma), and trains on the bilingual SFT data. The adapter weights are saved separately from the base model, enabling efficient storage and inference-time loading. Full training configuration details are provided in Section 4.5.

4.3.7 Stage 7: Base-vs-LoRA Benchmark

The final stage evaluates the fine-tuned LoRA adapter against the unadapted base model on held-out datasets. The benchmark script loads each model (base and base-plus-LoRA) in 4-bit inference mode, sends each held-out sample through the same prompt used during training, parses and validates the model output against the expected JSON schema, and computes a suite of metrics comparing model predictions to human-reviewed ground-truth labels. The benchmark protocol is detailed in Section 4.6.

4.3.8 End-to-End Traceability

A defining methodological property of this pipeline is end-to-end traceability. Every benchmark prediction can be traced back through the training data, the human-reviewed labels, the VLM pre-labels, and the original DCS screenshot. The pipeline produces versioned artifacts at each stage: capture manifests, pre-label JSONL files, Label Studio export files, SFT JSONL files, LoRA adapter weights, `train_summary.json` files,

benchmark predictions, and benchmark metric summaries. This chain of custody enables rigorous audit of any observed model behaviour, from a single fact misclassification up to aggregate benchmark metrics.

4.4 Dataset Curation

4.4.1 Dataset Overview

The visual fact data pipeline produced six datasets over successive capture and review cycles. Table 4.1 summarises each dataset's role, sample count, fact ontology version, review status, and export languages.

Table 4.1: Dataset inventory. All sample counts are from the corresponding `stats.json` files.

Dataset	Role	Samples
Run-001 (`vision_sft`)	Training source (8-fact v1)	180
Run-002 (`vision_sft_holdout_run002`)	Holdout (8-fact v1)	50
Run-002 newfacts (`vision_sft_holdout_run002_newfacts`)	Holdout (13-fact v2)	50
Run-003 (`vision_sft_run003`)	Training source (13-fact v2)	220
Run-004 (`vision_sft_holdout_run004_random`)	Holdout (13-fact v2)	100
Run-005 (`vision_sft_run005_composition_rebalance`)	Training supplement (13-fact v2)	122

Each dataset is fully human-reviewed: every sample has been inspected and corrected in Label Studio. Run-003 and Run-005 contain samples where the human reviewer left the summary field blank (82 and 25 samples, respectively, as recorded in the `missing_summary_count` field). These samples remain valid for training and evaluation because the facts array—the primary supervised target—is fully annotated for every fact in every sample.

4.4.2 Ontology Evolution: 8-Fact v1 to 13-Fact v2

The initial 8-fact v1 ontology (used in Run-001 and Run-002) defined eight facts: `fcs_page_visible`, `bit_root_page_visible`, `bit_page_failure_visible`, `right_ddi_fcsmc_page_visible`, `right_ddi_in_test_visible`, `fcs_bit_result_visible`, `ins_alignment_page_visible`, and `ins_go`.

The v1 ontology exhibited two structural weaknesses identified during the initial benchmark analysis. First, it mixed abstraction levels: some facts described low-level visual observations (e.g., `fcs_page_visible`), while others encoded higher-level procedural judgments (e.g., `ins_go`, which represented a completion signal for the INS alignment process rather than a directly observable visual feature). Second, several facts shared overlapping semantics: `bit_root_page_visible` and `bit_page_failure_visible` could overlap in certain cockpit states, and `right_ddi_fcsmc_page_visible` conflated the region identifier with the fact semantics.

The 13-fact v2 ontology (used in Run-003, Run-004, Run-005, and the current runtime) redesigned and expanded the fact set. The v2 facts are: `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `fcs_page_x_marks_visible`, `bit_root_page_visible`, `fcsmc_page_visible`, `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible` `hsi_page_visible`, `hsi_map_layer_visible`, `ins_grnd_alignment_text_visible`, and `ins_ok_text_visible`.

The v2 redesign addressed the v1 weaknesses through three strategies. First, the ambiguous `ins_go` fact was decomposed into two lower-level, directly observable facts: `ins_grnd_alignment_text_visible` (requiring literal GRND alignment block text) and `ins_ok_text_visible` (requiring clearly readable OK text near the alignment block). Second, region identifiers were removed from fact names (`right_ddi_fcsmc_page_visible` became `fcsmc_page_visible`), and page-visibility facts were added for the TAC and SUPT menus to cover the left DDI. Third, the FCS-MC state sequence was decomposed into a finer-grained four-state chain (`page`, `intermediate_result`, `in_test`, `final_go_result`), replacing the single `fcs_bit_result_visible` fact. Fourth, HSI-related facts were split into `hsi_page_visible` and `hsi_map_layer_visible`, and the FCS X-marks fact was separated from the FCS page visibility.

4.4.3 Training Recipe: Run-003 + Run-005x2

The primary training recipe for the 13-fact v2 models combines Run-003 (220 reviewed samples, bilingual export producing 220 English and 220 Chinese rows) with Run-005 composition rebalance data included twice (122 reviewed samples, bilingual export producing $122 \times 2 = 244$ English and $122 \times 2 = 244$ Chinese rows). The resulting training set contains 928 rows.

Run-005 was captured specifically as a composition rebalance dataset. Analysis of Run-003's fact-state distribution revealed severe class imbalance: several critical facts (e.g., `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_vi`

`ins_ok_text_visible`) had very few `seen` examples in the training data. Run-005 was designed with targeted cockpit-state transitions to oversample these underrepresented visual states.

The double-counting of Run-005 (Run-005x2) further amplifies the representation of these rare states in the training distribution. The input files for the training run are: `sft_en.jsonl` and `sft_zh.jsonl` from Run-003, plus two copies each of `sft_en.jsonl` and `sft_zh.jsonl` from Run-005, for a total of six input files.

No internal evaluation split was used: the training configuration sets `eval_rows = 0`. All evaluation is performed on independent holdout datasets.

4.4.4 Holdout Independence Verification

Holdout independence is a critical methodological requirement: the benchmark must measure cross-session generalisation, not in-distribution memorisation. Two holdout sets were constructed:

- **Run-002** (and its 13-fact re-annotation `run002_newfacts`): a new DCS capture session distinct from all training sessions. Contains 50 reviewed samples.
- **Run-004 random**: a further independent capture session with uniform random sampling of cockpit states. Contains 100 reviewed samples.

Exact overlap between each holdout set and the training data was verified to be zero (0). The overlap check compared image artifact paths and sample identifiers across training and holdout splits. No holdout sample appears in any training input file.

In contrast, the Run-001 benchmark used in early 8-fact v1 experiments is a **contaminated development set**: the same data pool from which training examples were derived was used for evaluation. Run-001 results are reported for regression analysis only and are explicitly distinguished from independent holdout results.

4.5 LoRA Fine-Tuning Configuration

4.5.1 Base Models

Two base VLM architectures were fine-tuned under the same data recipe to enable backbone comparison:

Qwen/Qwen3.5-9B-Base — a 9-billion-parameter vision-language model supporting multimodal input (image + text) and text generation. This is the primary experimental backbone.

google/gemma-4-31B — a 31-billion-parameter vision-language model from Google DeepMind. This is the supplementary comparison backbone, tested to verify whether the same data recipe generalises across model families.

Both models were loaded in 4-bit precision via Unsloth's `FastVisionModel.from_pretrained`. For Qwen, LoRA adapters were applied to both vision and language layers (`finetune_vision_layers=T` `finetune_language_layers=True`). For Gemma, LoRA was restricted to all linear modules (`lora_target_modules=all-linear`) and vision layers were not fine-tuned (`finetune_vision_layers=False`).

4.5.2 Training Stack

The training stack comprises three integrated components:

1. **Unsloth** — handles 4-bit model loading, LoRA injection, vision batch collation via `UnslothVisionDataCollator`, and FP16/BF16 automatic precision selection.
2. **PEFT (Parameter-Efficient Fine-Tuning)** — defines the LoRA adapter format, including rank, alpha, dropout, and target module selection.
3. **TRL SFTTrainer** — runs the supervised fine-tuning loop with configurable batch size, gradient accumulation, epochs, learning rate, warmup, and weight decay.

4.5.3 Hyperparameter Configuration

Table 4.2 lists the exact hyperparameters used for the primary training run (Run-003 + Run-005x2), matched across both backbones. All values are drawn from the `train_summary.json` artifacts.

The final training loss values (0.2156 for Qwen, 0.0474 for Gemma) should not be compared directly across backbones, because tokenizer behaviour, chat-template structure, target-sequence distribution, and model parameterisation differ. The epoch count of 4 was selected as a compromise for small-data domain adaptation: with only 928 training examples, fewer epochs may not reliably teach the model the fixed JSON schema and cockpit fact boundaries, while more epochs risk overfitting to the training sessions.

Table 4.2: LoRA fine-tuning hyperparameters, matched across Qwen3.5-9B and Gemma-4-31B.

Parameter	**Qwen3.5-9B**	**Gemma-4-31B**
`max_seq_length`	4096	4096
`num_train_epochs`	4.0	4.0
`learning_rate`	2×10^{-4}	2×10^{-4}
`per_device_train_batch_size`	1	1
`gradient_accumulation_steps`	4	4
`lora_r`	16	16
`lora_alpha`	16	16
`lora_dropout`	0.0	0.0
`seed`	3407	3407
`load_in_4bit`	True	True
`warmup_ratio`	0.1	0.1
`weight_decay`	0.01	0.01
`train_rows`	928	928
`eval_rows`	0	0
Backbone-specific		
Vision LoRA	enabled	disabled
LoRA target modules	vision + language + attention + MLP	all-linear
Chat template	Qwen default	gemma-4
GPU memory utilisation	0.6	0.95
Training outcomes		
Train runtime (s)	6419.16	8049.65
Train loss (final)	0.2156	0.0474

The LoRA rank $r = 16$ and alpha $\alpha = 16$ provide moderate adaptation capacity: sufficient to learn cockpit layout patterns and JSON formatting, but not as large as a high-capacity adapter ($r \geq 64$) that might overfit the small dataset more aggressively. Dropout was set to zero for the primary run, prioritising learnability over regularisation in this first adaptation experiment.

4.5.4 Bilingual SFT Strategy

The bilingual SFT strategy produces two training rows per reviewed sample: one with English system and user instructions, and one with Chinese instructions. Both rows share the same image (base64-encoded) and the same assistant target (ground-truth facts JSON). The English assistant target is reused for the Chinese rows; the model's task is to extract visual facts from the image, not to generate language-specific natural-language descriptions.

This approach increases instruction-following robustness across languages without requiring additional human annotation effort. It does not, however, substitute for additional cockpit-state screenshots or hard-negative visual data.

4.5.5 Composition Rebalance Strategy

The composition rebalance strategy (Run-005) addresses the class imbalance observed in the primary training source (Run-003). In Run-003, several facts had extremely low positive (`seen`) counts: `ins_ok_text_visible` had only 12 `seen` examples out of 220 samples (5.5%); `fcs_page_x_marks_visible` had 16 (7.3%); `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, and `fcsmc_final_go_result_visible` each had 18 (8.2%).

Run-005 captured 122 additional samples targeting cockpit states in which these rare facts are `seen`. After Run-005 augmentation, the combined training set provides improved coverage of previously underrepresented facts: `supt_page_visible` (from 20 to 54 combined seen), `fcsmc_page_visible` (from 55 to 111), `hsi_page_visible` (from 106 to 215), and `ins_grnd_alignment_text_visible` (from 58 to 130).

The Run-005 data is included twice in the training recipe (Run-005x2), further amplifying the signal for these underrepresented visual states.

4.6 Benchmark Protocol

4.6.1 Metrics

The benchmark compares model predictions against human-reviewed ground-truth labels using the following metrics:

`json_valid_rate` — the proportion of samples for which the model output is parseable as a JSON object. A failure here indicates that the model produced non-JSON text (e.g., markdown, natural-language commentary, or malformed braces).

`schema_valid_rate` — the proportion of samples for which the parsed JSON conforms to the expected schema: a top-level object containing only `summary` and `facts` keys, a `facts` array where each entry contains exactly `fact_id`, `state`, and `evidence_note`, all 13 fact IDs present exactly once, and all states in `{seen, not_seen, uncertain}`.

`fact_accuracy` — three-class accuracy over all facts in all samples, computed as the proportion of individual fact predictions that match the ground-truth label.

`macro_f1` — macro-averaged F1 score across the three states (`seen`, `not_seen`, `uncertain`), computed per fact and then averaged across all 13 facts.

`seen_f1` — F1 score for the positive `seen` class, computed per fact and then averaged across all 13 facts. This is a particularly important metric because missing a `seen` state (false negative) or hallucinating a `seen` state (false positive) directly affects downstream procedure-state reasoning.

`sample_exact_match` — the proportion of samples for which all 13 facts match the ground-truth labels exactly. A single fact misclassification counts as a sample-level failure.

`critical_false_positive_count` — the count of errors in which a fact classified as *critical* (a subset of 6 facts: `fcs_page_x_marks_visible`, `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`, `ins_grnd_alignment_text_visi`, `ins_ok_text_visible`) is predicted as `seen` when the ground truth is `not_seen` or `uncertain`.

Critical false positives are tracked separately because they represent the most consequential error type in the tutoring context: incorrectly reporting a completion-critical

state as observed when it is not. A tutor that erroneously believes the INS alignment is complete (based on a `ins_ok_text_visible` false positive, for example) may skip essential guidance, potentially leading to procedural errors.

The benchmark also computes per-fact confusion matrices and per-fact precision, recall, and F1 scores, which are stored in `fact_scores.csv` alongside automatically generated charts (overall accuracy, per-fact macro F1, per-fact seen F1, critical false positives, and per-fact confusion matrices).

4.6.2 Base vs. LoRA Comparison Methodology

The benchmark protocol compares two model configurations against the same ground-truth labels:

1. **Base model**: the unadapted base VLM loaded in 4-bit inference mode without any LoRA adapter.
2. **LoRA model**: the same base VLM loaded in 4-bit inference mode with the LoRA adapter weights loaded via PEFT's `PeftModel.from_pretrained`.

Both configurations receive the identical prompt, the identical images, and the identical generation parameters: temperature = 0.0 (greedy decoding), `max_new_tokens` = 1024, and seed = 3407. The benchmark runs each sample independently, serialises predictions to JSONL, and computes metrics without any post-hoc filtering or exclusion.

The comparison script (`comparison.json`) computes delta metrics (LoRA minus Base) and an automated recommendation flag (`recommend_simtutor_chain_validation`) that activates when the LoRA adapter simultaneously improves JSON validity, fact accuracy, seen F1, and does not increase critical false positives.

4.6.3 Holdout Definitions and Benchmark Categories

Each benchmark run is assigned a `benchmark_kind` that determines how its results should be interpreted:

`contaminated_dev_set` — the evaluation set is drawn from the same data pool as the training set. Results indicate whether the model learned the schema and in-distribution visual boundaries but cannot support generalisation claims. Used for Run-001 benchmarks.

heldout_new_session — the evaluation set is a completely independent capture session with zero overlap with any training data. Results support cross-session generalisation claims. This is the primary evaluation category. Run-002 newfacts and Run-004 random benchmarks fall into this category.

The distinction between contaminated and heldout benchmarks is maintained rigorously throughout the benchmark infrastructure and is reflected in the results chapter (Chapter 6).

4.6.4 Benchmark Artifacts

Each benchmark run produces a standardised directory of artifacts:

- `benchmark_summary.json` — top-level summary with comparison deltas and chart paths.
- `metrics_base.json` / `metrics_lora.json` — full metric dictionaries, including per-fact scores and confusion matrices.
- `predictions_base.jsonl` / `predictions_lora.jsonl` — per-sample model outputs, one JSON object per line.
- `errors_base.jsonl` / `errors_lora.jsonl` — individual prediction errors with gold labels, predicted labels, and raw model text.
- `comparison.json` — delta metrics and recommendation flag.
- `fact_scores.csv` — CSV table of per-fact, per-model metrics.
- `report.md` — human-readable markdown summary.
- `charts/` — automatically generated charts: overall accuracy, sample exact match, per-fact macro F1, per-fact seen F1, critical false positives, and per-fact confusion matrices for base and LoRA models.

These artifacts constitute the full evidence chain for the technical results reported in Chapter 6. Every metric, chart, and error record is reproducible by re-running the benchmark script with the same reviewed JSONL and LoRA adapter weights.

4.7 Summary

This chapter has described the four methodological pillars of the SimTutor system: telemetry-grounded procedure runtime, knowledge grounding with BM25 retrieval and source policy, the seven-stage visual fact data pipeline from screenshot capture to benchmark, and the LoRA fine-tuning protocol with matched hyperparameters across two backbone architectures. The methodology is characterised by end-to-end traceability—every benchmark metric can be traced through training data, human-reviewed labels, and original screenshots—and by a strict separation between contaminated development benchmarks and independent holdout benchmarks. These methodological choices directly support the claims and results presented in Chapters 6 and 7.

5 Implementation

This chapter describes the concrete implementation of the SimTutor system whose design was presented in Chapter 3. It covers the technology stack, the core domain modules, the telemetry and DCS integration pipeline, the structured help generation subsystem, the VLM visual fact extraction runtime, the replay and logging infrastructure, and the CLI and live runtime entrypoints. Methodology-level details—the VLM adaptation protocol, dataset curation, and benchmark design—appear in Chapter 4.

5.1 Technology Stack

The SimTutor Python codebase is managed with Poetry[1] and requires Python 3.10 or later. Core library dependencies are kept minimal: `httpx` (v0.28+) for HTTP client functionality, `PyYAML` (v6+) for all declarative configuration, `jsonschema` (v4.23+) for Draft 2020-12 JSON Schema validation of model outputs and telemetry frames, and `Pillow` (v12+) for image handling in the VLM pipeline. Testing relies on `pytest` (v7.4+) and `pytest-cov` (v5.0+) with branch coverage enabled.

The DCS Lua-side integration comprises five scripts deployed to the DCS `Saved Games` directory: `Export.lua` bootstraps the DCS-BIOS export protocol; `VRHilite.lua` and `SimTutorHighlight.lua` listen on UDP sockets for highlight, clear, and pulse commands and render them as cockpit overlays via `net.dostring_in`; `SimTutor.lua` and its companion `SimTutor Function.lua` handle composite-panel screenshot capture and the DCS-BIOS hub integration.

Model serving is provider-agnostic. The primary path uses an OpenAI-compatible API endpoint (intended for vLLM, llama.cpp, or TGI servers), configured via the environment variable `SIMTUTOR_MODEL_PROVIDER=openai_compat` together with a base URL and API key. An Ollama fallback (`SIMTUTOR_MODEL_PROVIDER=ollama`) provides local compatibility without an API key. A deterministic stub (`SIMTUTOR_MODEL_PROVIDER=stub`) returns fixed responses for testing without any external model service. Multimodal sup-

[1] Version 0.1.0, declared in `pyproject.toml` under the package name `simtutor-core`.

port is enabled by setting `SIMTUTOR_MODEL_ENABLE_MULTIMODAL=1`, which causes the OpenAI-compatible adapter to include base64-encoded images in the request payload.

LoRA fine-tuning of vision-language models uses the Unsloth library with PEFT and TRL, targeting Qwen3.5-9B-Base as the primary backbone and Gemma-4-31B as a supplementary line. Pre-labeling and human review of vision fact annotations are conducted in Label Studio.

5.2 Core Domain Layer

The core layer implements all domain logic that is independent of any particular simulator, model provider, or external service. It is organised into approximately fifteen modules, each with a narrow responsibility.

5.2.1 Procedure Engine

The `ProcedureEngine` class in `core/procedure.py` implements the step state machine described in Section 3.6. Each step is represented by a `StepState` dataclass holding a step identifier and one of four statuses defined in the `StepStatus` enumeration: `PENDING`, `ACTIVE`, `DONE`, or `BLOCKED`. The engine enforces that at most one step is `ACTIVE` at any time. Five operations are exposed: `activate_next` advances the next `PENDING` step to `ACTIVE`; `complete_active` transitions the active step to `DONE`; `block_active` marks it `BLOCKED`; `rewind` returns a completed step to `ACTIVE`; and `check_timeout` tests whether the active step has exceeded a configurable wall-clock limit. All state transitions emit internal event records with ISO-8601 timestamps and optional reason strings; these events are later propagated to the JSONL event store.

5.2.2 Gating Rule Engine

The gating engine in `core/gating.py` evaluates precondition and completion gates expressed in a small domain-specific language. Four operators are implemented: `var_gte` (telemetry value threshold), `arg_in_range` (numeric interval membership), `flag_true` (boolean flag with string coercion), and `time_since` (elapsed wall-clock time since a tagged event). Each rule returns a `RuleResult` dataclass carrying a boolean status, a failure path in dot-notation, and a human-readable reason. The evaluator includes explicit unknown-value handling: the sentinel strings `"unknown"`, `"unk"`, `"missing"`, `"n/a"`, and `"na"` cause rule failure with a diagnostic message rather than silent treatment as

`False`. Rules short-circuit on first failure; the index and reason of the first unsatisfied rule feed into the deterministic step inference logic.

5.2.3 Scoring Engine

The scoring module in `core/scoring.py` implements a simple event-log scorer following the error taxonomy defined in the procedure pack. It counts two classes of error: omissions (`OM`), defined as canonical steps up to the maximum observed step that were never completed, and state violations (`SV`), identified by `"state_violation"` tags in observation payloads. A critical-step multiplier is applied to omissions of steps marked `critical` in the step registry. Three additional categories (`CO`, `OR`, `PA`) have their counters and base weights defined in the taxonomy but default to zero in the current v1 scoring implementation. The total error score is computed as a ceiling-rounded weighted sum.

5.2.4 Vision Fact Ontology and State Management

The vision facts module in `core/vision_facts.py` centralises the 13-fact v2 ontology as the `VISION_FACT_IDS` tuple, containing identifiers such as `tac_page_visible`, `fcsmc_final_go_result_visible`, and `ins_ok_text_visible`. Configuration is loaded from `vision_facts.yaml` through a filesystem-stat-cached, LRU-cached loader (`load_vision_facts_c` which normalises fact specifications (sticky flag, TTL in milliseconds, intended regions, associated step IDs) and step bindings (`all_of` and `any_of` conditions).

The core merging logic in `merge_vision_fact_observation` implements three key semantics. First, *sticky preservation*: a fact whose prior state is `seen`, whose `sticky` flag is `True`, and whose TTL has not expired will not be overwritten by a subsequent `not_seen` or `uncertain` observation—only a later `seen` or expiry can replace it. Second, *TTL-based expiry*: `prune_expired_facts` discards any fact whose `expires_at_wall_ms` precedes the current wall clock. Third, *step binding satisfaction*: `facts_satisfy_step_binding` checks that all facts in the `all_of` list are `seen` and that at least one fact in the `any_of` list (if defined) is `seen`.

For the `fcsmc_final_go_result_visible` fact (step S18), an additional coercion layer checks the `result_kind` field and the `evidence_note` text against a set of regular expressions (e.g., `"MC1: GO"`, `"FCSA: GO"`). A `seen` state is only retained if the classification yields `"final_go"`; intermediate states such as `"intermediate_go"`, `"in_test"`, or `"not_ready"` are coerced to `"not_seen"`, and anything else is coerced to `"uncertain"`.

5.2.5 Dynamic HelpResponse Schema

The module `core/llm_schema.py` generates the JSON Schema for structured help responses at runtime, not from a static file. The function `build_help_response_schema` receives three lists of enum values—step identifiers from the step registry (or pack configuration), overlay target names from the UI map, and error category codes from the taxonomy—and constructs a Draft 2020-12 schema with four required top-level keys: `diagnosis`, `next`, `overlay`, and `explanations`. A particularly detailed part of the schema is the `overlay.evidence` array: for each overlay target, a conditional subschema (`if/then`) requires that the evidence array contains at least one item referencing that target, thereby enforcing that the model cannot request an overlay without citing the evidence source from which it inferred the target. The schema also requires each evidence item to carry a `type` field restricted to the set `{"var", "gate", "rag", "delta", "visual"}`, a non-empty `ref` and `quote`, and a `grounding_confidence` between 0.0 and 1.0. The computed schema is cached keyed on the filesystem modification times and sizes of its source files, so schema regeneration is avoided during long-running sessions.

5.2.6 Data Contracts

Two versions of data contracts coexist. The v1 types in `core/types.py` define four core dataclasses used throughout the runtime: `Observation` (timestamped simulator state with optional procedure hint and tags), `TutorRequest` (learner-initiated help intent with an observation reference and context dictionary), `TutorResponse` (tutor output with status, message, overlay actions, and explanations), and `Event` (general-purpose log entry with session identifier, event kind, versioned payload, and extensible metadata). All four carry UUID-based identifiers, ISO-8601 wall-clock timestamps, and a `to_dict` serialisation method.

The v2 types in `core/types_v2.py` extend the contract space with simulator-specific structures: `DcsObservation` (raw DCS-BIOS frame with sequence number and cockpit dictionary), `TelemetryFrame` (enriched state with normalised `vars`, BIOS snapshot, and cockpit arguments), `VisionObservation` (screenshot metadata with frame identifier, layout identifier, capture timestamp, and image URI), `VisionFact` (individual visual fact with identifier, three-valued state, source frame reference, TTL, and evidence note), and `VisionFactObservation` (collection of facts returned by the extractor, carrying a trigger timestamp and frame identifier list).

5.2.7 Additional Core Modules

Several supporting core modules complete the domain layer. `core/vars.py` implements `VarResolver`, a telemetry variable resolver that evaluates restricted Python expressions (numeric arithmetic, boolean logic, comparisons, and a `num()` coercion function) to project raw BIOS data onto normalised state variables, while tracking `vars_source_missing` for key-level observability. `core/knowledge.py` provides a `BM25Retriever` class implementing standard BM25 term weighting with Okapi-style IDF, used by the local knowledge adapter for document snippet retrieval. `core/overlay.py` defines the `OverlayPlanner` class, which maps abstract target names (e.g., `battery_switch`) to DCS-internal point codes using the procedure pack's UI map, and produces `OverlayIntent` objects with one of three intents: `highlight`, `clear`, or `pulse`. `core/step_signal_metadata.py` provides observability status normalisation and `compute_requires_visual_confirmation`, which determines whether a step requires visual evidence based on its `observability` field and `evidence_requirements` list. `core/help_cycle_audit.py` defines the canonical set of help-cycle audit fields (`generation_mode`, `vision_used`, `frame_id`, `sync_delta_ms`, etc.) and a normalisation function used consistently by both the live runtime and the replay path.

5.3 Telemetry and DCS Integration

The telemetry integration pipeline operates in four stages, each implemented by a dedicated adapter module.

Raw DCS-BIOS ingestion. The `DcsBiosRawReceiver` class opens a UDP socket and listens for DCS-BIOS binary export frames. Raw frames are decoded as UTF-8 JSON and written directly to a JSONL file without processing, enabling offline replay of BIOS data independently of the live runtime. The companion `DcsBiosReceiver` class provides a higher-level interface that parses BIOS payloads, enforces monotonic sequence numbers, merges full-state deltas into an accumulated state dictionary, and produces `Observation` objects for consumption by the tutoring runtime.

BIOS-to-UI mapping. The `BiosUiMapper` class loads `bios_to_ui.yaml` from the procedure pack, mapping DCS-BIOS key names to abstract UI target identifiers. This mapping enables the delta aggregation pipeline and the deterministic step inference module to translate raw telemetry changes into semantically meaningful UI references.

Variable resolution and enrichment. `enrich_bios_observation` in `adapters/telemetry_pipe` applies the `VarResolver` to a `TelemetryFrame`, producing normalised key–value pairs

in the `vars` dictionary. The resolver projects from four reference roots (`bios`, `lo`, `cockpit_args`, `vars`) via expressions defined in `telemetry_map.yaml`. Variables whose source references are absent or whose expression evaluates to `None` are tracked in `vars_source_missing`, providing downstream components with key-level observability.

Delta sanitisation and aggregation. The `DeltaSanitizer` class (in `adapters/delta_sanitizer` applies a configurable `DeltaPolicy` loaded from `delta_policy.yaml` to suppress spurious state changes. The policy specifies per-variable numeric-change thresholds, debounce windows, and filtering rules. Validated changes are emitted as `SanitizedDelta` records carrying kept and dropped change counts with per-reason breakdowns. The `DeltaAggregator` (in `adapters/delta_aggregator.py`) accumulates sanitised deltas over a configurable time window and compresses them into a `DeltaSummary` containing recent UI targets (mapped through `BiosUiMapper`), top-k key changes, and dropped-change statistics. These summaries are included in the help-cycle prompt context and in the recent-actions ring buffer used by the live runtime.

Telemetry frame schema. The v2 telemetry frame schema (`simtutor/schemas/v2/telemetry_fr` defines the canonical structure of enriched telemetry records, including wall-clock timestamp, session identifier, BIOS snapshot, cockpit arguments, normalised variables, and optional low-level outputs. All live and replay paths validate telemetry frames against this schema at the point of emission.

The entire telemetry pipeline runs as a continuous background thread in the live runtime, independent of any specific help cycle.

5.4 Structured Help Generation

The help generation subsystem transforms a tutor request and its assembled context into a validated, executable tutor response. It proceeds through six stages.

5.4.1 Prompt Construction

The function `build_help_prompt_result` in `adapters/prompting.py` assembles the full prompt sent to the language model. It includes: the current simulator state (`vars`), sanitised recent deltas, candidate procedure steps with their observability status and visual confirmation requirements, the deterministic step hint and its missing conditions list, the overlay target allowlist, BM25-retrieved knowledge snippets (up to five, each truncated to 220 characters), and, when available, the current vision fact summary. The

prompt is subject to a hard character cap (9000) with an estimated token limit of 2400. Interaction policy rules for switch position, knob rotation, and button click conventions are included in either Chinese or English.

5.4.2 Model Adapter

The model adapter layer supports three backends through a common interface. `OpenAICompatModel` (in `adapters/openai_compat_model.py`) posts chat-completion requests to an OpenAI-compatible endpoint, appending `/v1/chat/completions` to the configured base URL. It requires an API key, supports configurable timeout and max-token limits, and can be toggled to multimodal mode. `OllamaModel` (in `adapters/ollama_model.py`) is a zero-dependency adapter for local Ollama servers. `ModelStub` (in `adapters/model_stub.py`) returns a fixed deterministic response for testing, enabling the full help-generation pipeline to be exercised without an external model service. All adapters return a raw text response through a unified call interface.

5.4.3 JSON Extraction with Minimal Repair

Model responses are not guaranteed to contain only valid JSON. The module `adapters/json_extract.` implements a `parse_first_json` function with a deliberately narrow repair set, limited to four transformations: (1) removal of balanced Markdown code fences (triple-backtick JSON blocks), (2) removal of leading `<think>...</think>` blocks, (3) dropping non-JSON prefix text before the first `{` or `[`, and (4) dropping suffix text after the closing brace or bracket. Any transformation outside this set is rejected. The extraction uses a stack-based parser that tracks string quoting and escape sequences to correctly locate balanced JSON delimiters. Each extraction returns a `JsonExtractionResult` recording whether repairs were applied and enumerating the specific repair reasons.

5.4.4 Schema Validation

The extracted JSON object is validated against the runtime-generated HelpResponse schema (Section 5.2) using `Draft202012Validator` from the `jsonschema` library. Validation errors are reported with JSON Path locations derived from the validator's error path. The schema enforces four categories of constraint: (1) required top-level keys, (2) enum membership for step identifiers and error category codes, (3) overlay target uniqueness and allowlist membership, and (4) evidence-item presence for every overlay target mentioned.

5.4.5 Response Mapping and Evidence Grounding

The function `map_help_response_to_tutor_response` in `adapters/response_mapping.py` translates the validated HelpResponse into a runtime `TutorResponse`. It resolves overlay targets to DCS-internal element identifiers through the `OverlayPlanner`, which consults `ui_map.yaml`. The mapping function also validates that every overlay target cited in the response has at least one evidence reference whose prefix (`VARS.`, `GATES.`, `VISION_FACTS.`, `RAG_SNIPPETS.`, or `DELTA_KEYS.`) matches a key actually present in the help-cycle context. Targets without grounding evidence are rejected, and the rejection is recorded in the response metadata. Explanations and diagnosis fields are passed through as-is. The `TutorResponse` status is set to `"ok"` on full success and to `"partial"` or `"fallback"` when mapping failures occur.

5.4.6 Deterministic Fallback

When no model is available, model output fails JSON extraction or schema validation, or response mapping produces an empty or invalid result, the system falls back to deterministic step inference. The class `StepInferenceResult` (in `adapters/step_inference.py`) examines the active step, evaluates unsatisfied gating rules, identifies missing conditions, maps condition references to likely UI targets via a hard-coded lookup table, and produces a text hint restricted to the currently active step. By default, deterministic fallback does not trigger overlay highlights unless a local rule can prove a unique, valid target given the current telemetry state. The fallback result is logged with the reason for fallback in the help-cycle audit metadata.

5.4.7 Action Execution Safety

The `OverlayActionExecutor` in `adapters/action_executor.py` enforces three safety constraints on all actions before dispatching them to DCS via UDP. First, only actions of type `"overlay"` are executable—no control injection, no key emulation, no click simulation. Second, every overlay target is checked against the UI target allowlist derived from the pack's `ui_targets` field; targets not in the allowlist are rejected and logged. Third, at most one target may be highlighted at a time (`max_targets=1`). When a new highlight is requested, a prior active highlight is automatically cleared to prevent visual clutter. The `pulse` intent uses a blocking sleep (`ttl_s`) before dispatching a clear command, which blocks the calling thread but guarantees complete pulse delivery. A configurable acknowledgement receiver (`DcsOverlayAckReceiver`) can optionally verify

delivery by listening for UDP acknowledgement messages from the Lua-side overlay script.

5.5 VLM Visual Fact Extraction

The VLM visual fact extraction runtime is implemented as a core, independently operable subsystem that communicates with the tutoring runtime through structured observations.

5.5.1 Capture Trigger and Frame Selection

When the active step is vision-priority (S08, S15, or S18) and a help request arrives, a capture request is dispatched to DCS via UDP using `build_capture_request_payload` in `adapters/vision_capture_trigger.py`. The Lua-side `SimTutor.lua` script renders the three primary cockpit displays (left DDI, AMPCD, right DDI) into a single composite PNG image. The Python-side `BufferedVisionSession` (in `adapters/vision_sync.py`) maintains a sliding buffer of recent vision observations and performs frame selection: by default, it retrieves both a pre-trigger frame (the most recent frame immediately before the trigger event within a configurable sync window of 250 ms) and the trigger frame itself. If a pre-trigger frame is unavailable within the window, the session falls back to trigger-frame-only mode. Frame staleness and sync miss reasons are recorded in the frame payload metadata.

5.5.2 Multimodal Payload Construction

The selected candidate frames are converted into a multimodal chat-completion payload by `build_multimodal_image_contents` in `adapters/openai_compat_multimodal.py`. Local frame files under a configurable root directory are read and base64-encoded as `data:` URLs with MIME type detection. Remote URLs and inline data URLs are rejected. A configurable maximum file size guards against accidentally large images. The function produces a list of `image_url` content blocks, each tagged with the frame's role (`"pre_trigger"` or `"trigger"`), which are inserted alongside the text prompt into the messages array of the OpenAI-compatible request.

5.5.3 13-Fact Prompt

The prompt for visual fact extraction is built by `build_vision_fact_prompt` in `adapters/vision_fac` It enumerates all 13 configured facts, describes the three-region composite-panel layout (top: left DDI; middle: AMPCD; bottom: right DDI), and provides detailed decision-boundary instructions for each fact: for example, `tac_page_visible` requires the actual TAC page content, not merely a TAC menu label; `fcsmc_final_go_result_visible` requires final GO results for all four channels (MC1, MC2, FCSA, FCSB); `ins_ok_text_visible` requires clearly readable `OK` text near the INS alignment block. The prompt is available in both Chinese and English via a `lang` parameter.

5.5.4 Model Response Sanitisation

The raw model output is parsed as JSON and sanitised by `_sanitize_model_response` in `adapters/vision_fact_extractor.py`. The sanitisation function strips unknown top-level keys, rejects fact array entries that are not mappings, rejects entries with missing required fields (`fact_id`, `state`, `evidence_note`), and rejects fact entries whose `fact_id` does not belong to the valid fact ID set defined in the configuration. Only three fact states are accepted: `"seen"`, `"not_seen"`, and `"uncertain"`. A dynamic JSON Schema is generated on-the-fly from the allowed fact IDs using `_vision_fact_response_schema`, and the sanitised output is validated against it via `Draft202012Validator`.

5.5.5 Fact Merging and Backend Compatibility

The sanitised facts are merged into the runtime visual fact snapshot through `merge_vision_fact_obser` (Section 5.2), which applies sticky preservation and TTL-based expiry before updating the snapshot. The VLM backend compatibility layer in `adapters/openai_compat_multimodal.py` handles provider-specific error conditions, including detection of `400` responses indicating multimodal or JSON-schema-unsupported backends, and distinguishes between genuine errors and the `null` payload sent by DashScope for multimodal responses. When the VLM is unavailable or the multimodal request fails, the help cycle proceeds with telemetry and knowledge only, recording the reason in the vision fallback metadata.

5.5.6 VR Compatibility

The current VLM runtime extracts facts from the composite-panel layout (left DDI, AMPCD, right DDI). Composite-panel capture has been validated for compatibility in

VR mode (Quest 3), using VR_MIRROR and VR_allow_MFD_out_of_HMD configurations. The vision port abstraction supports alternative frame sources without changes to the extraction or merging logic.

5.6 Replay and Logging Infrastructure

5.6.1 JSONL Event Store

The `JsonlEventStore` class in `core/event_store.py` is the sole persistence mechanism for runtime events. It implements an append-only writer with immediate `flush` after each write, ensuring crash-safe logging. The store supports context-manager usage (`__enter__` / `__exit__`) and provides a static `load` method that reads an entire JSONL file into memory as a list of dictionaries. Event serialisation accepts both `Event` dataclass instances and plain dictionaries, delegating to `Event.to_dict()` when the former is supplied.

5.6.2 Replay and Validation

The replay module in `simtutor/runner.py` provides two entry points. `run_simulation` drives a full simulation using a `MockEnvAdapter` that replays pre-recorded observation sequences, coordinated with a `ProcedureEngine` instance, and writes the resulting event log to a timestamped JSONL file. `replay_log` validates an existing event log by replaying it against a procedure pack: it reconstructs the step state machine, verifies that each event's step status transitions are legal (e.g., a step cannot be completed before it is activated), and returns a boolean success flag together with a diagnostic message. Replay validation includes checks for monotonic sequence numbers and wall-clock timestamps in telemetry traces.

The replay evaluation infrastructure in `simtutor/replay_eval.py` extends replay to support suite-driven evaluation: a YAML suite file specifies a pack path, a set of test cases, model configuration, and optional vision frame directories, enabling systematic regression replay at the telemetry level and for vision sidecar scenarios.

5.6.3 Interaction Metrics

The `compute_interaction_metrics` function in `core/interaction_metrics.py` derives behavioural metrics from event logs. It computes: stall time per step (total and

maximum, derived from `step_activated` to `step_completed` intervals), help request count, LLM trigger count (tutor responses with model metadata), UI click count and failure count, and highlight/clear request and acknowledgement counts. The output is an `InteractionMetrics` dataclass with a `to_dict` serialisation method.

5.6.4 Current Limitations

The logging and replay infrastructure has been exercised with synthetic regression scenarios (mock environments, pre-recorded observation sequences, and offline BIOS captures). End-to-end replay of full multimodal sessions with synchronised vision frames from live human-in-the-loop runs has not yet been validated.

5.7 CLI and Runtime Entrypoints

5.7.1 CLI Commands

The `simtutor` package provides a command-line interface through `simtutor/__main__.py`. The following commands are implemented:

`run`. Executes a simulation using a `MockEnvAdapter` with a pre-recorded scenario file and writes the event log to a timestamped JSONL file.

`replay`. Validates an existing event log against a procedure pack, checking step ordering and state-machine consistency.

`score`. Scores an event log using the scoring engine, producing omission counts, state-violation counts, and a total error score.

`record-vlm`. Runs VLM fact extraction on a set of vision observations and records the predictions for benchmarking.

`replay-bios`. Replays a raw DCS-BIOS capture (`dcs_bios_raw.jsonl`) through the telemetry pipeline and optionally invokes the help generation loop, producing a new timestamped event log.

`validate`. Validates JSONL files against a named schema from the schema registry (e.g., `dcs_bios_frame`, `telemetry_frame`, `vision_fact_observation`).

All commands accept a `-model-provider`, `-model-name`, `-model-base-url`, and `-model-timeout-s` argument, allowing model configuration to be overridden at the command line. The `-lang` flag switches between Chinese (`zh`) and English (`en`) prompt and output languages.

5.7.2 Live Runtime Orchestration

The module `live_dcs.py` is the primary live runtime entrypoint. It orchestrates the following continuous loop:

1. A `DcsBiosRawReceiver` or `DcsBiosReceiver` ingests UDP frames from DCS.
2. Each raw frame is enriched through `enrich_bios_observation` to produce normalised state variables.
3. When a help trigger is received (hotkey via `WindowsGlobalHelpTrigger` or UDP message), a help cycle begins:
   - The deterministic step hint is computed via `infer_step_id`, evaluating pack gates against current telemetry.
   - If the active step requires visual confirmation, a vision capture trigger is sent and the VLM fact extractor is invoked.
   - A BM25 grounding query is built and knowledge snippets are retrieved.
   - The full prompt is assembled and sent to the model adapter.
   - The model response is parsed, validated, and mapped to a `TutorResponse`.
   - Overlay actions are executed via `OverlayActionExecutor`, with allowlist and cardinality enforcement.
   - The complete help cycle (request, response, actions, vision facts, fallback reason if any) is logged as `Event` records.
4. The loop continues until terminated.

The live runtime supports both real-time DCS operation and replay of pre-recorded BIOS captures through a shared code path, controlled by the `-replay-bios` flag. All threading (BIOS receiver, overlay acknowledgement receiver, help cycle processing) is managed via Python standard-library threading primitives with queue-based inter-thread communication.

5.7.3 DCS Lua Scripts

Three categories of Lua scripts execute inside the DCS process:

State export. `Export.lua` loads DCS-BIOS and the SimTutor integration module, establishing the state-export chain.

Overlay rendering. `VRHilite.lua` binds a UDP socket on port 7778 and listens for plain-text overlay commands (`HIGHLIGHT <pnt>`, `CLEAR`, `PULSE <pnt> <ttl_s>`). Highlights are rendered via DCS's `a_cockpit_highlight` / `a_cockpit_remove_highlight` API accessed through `net.dostring_in`. `SimTutorHighlight.lua` provides an alternative rendering path for SimTutor-specific highlight contexts.

SimTutor integration. `SimTutor.lua` loads `SimTutor Function.lua` (which contains the capture and export logic) and installs the export chain. `SimTutorDcsBiosHub.lua` bridges SimTutor-specific state queries to the DCS-BIOS data model.

5.7.4 Model Provider Configuration

Model access is configured through environment variables with a unified naming convention (Section 3.2). The live runtime validates all required configuration at startup, emitting a non-sensitive summary (provider, model name, timeout, language, base URL) and rejecting the session with a clear error message if any required variable is missing. API keys are never included in log output or exception messages, enforced by the `redact_sensitive_text` function in `core/security.py`.

6 Evaluation

This chapter presents the empirical evidence gathered during the development of the visual fact extraction subsystem and the runtime infrastructure that supports it. It is divided into two clearly delineated parts. Part A (Section 6.1) reports technical validation results that are backed by benchmark data, dataset statistics, and training artifacts currently present in the repository. Part B (Section 6.2) describes the planned human-subject evaluation that constitutes the original research vision of this thesis but has not yet been conducted; it is included here to document the evaluation design against which future work can be measured.

6.1 Current Technical Results

The technical evaluation addresses a single core question: can a LoRA-fine-tuned vision-language model (VLM) reliably extract structured visual facts from F/A-18C cockpit screenshots, and does the resulting adapter substantially outperform the untuned base model? The question is assessed through a series of benchmark comparisons on data that the model has never seen during training (holdout sets with verified zero overlap). The metrics reported below are computed by the automated benchmark harness described in Section **??** and are drawn directly from the benchmark artifacts stored in the repository.

6.1.1 Dataset and Training Summary

The visual fact extraction task evolved across two generations of fact ontology. The first generation (v1) defined 8 visual facts and was used to establish the initial training and evaluation pipeline. The second generation (v2) expanded the ontology to 13 visual facts, aligning it with the runtime visual contract defined in `packs/fa18c_startup/vision_facts.yaml`. The 13 facts cover panel-level page visibility (e.g., `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`), fine-grained state indicators within panels (`fcsmc_intermediate_result_visibl` `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`), and small-text recognition targets (`ins_grnd_alignment_text_visible`, `ins_ok_text_visible`).

Table 6.1 summarises the six annotated datasets present in the repository, each with human-reviewed ground-truth labels.

Table 6.1: Summary of annotated visual fact datasets. Bilingual export produces separate EN and ZH samples from the same reviewed tasks. Holdout sets are English-only.

Dataset	Run	Facts	Tasks	Bilingual	Role
`vision_sft`	Run-001	8	180	yes	v1 train
`vision_sft_holdout_run002`	Run-002	8	50	no	v1 hold
`vision_sft_holdout_run002_newfacts`	Run-002	13	50	no	v2 hold
`vision_sft_run003`	Run-003	13	220	yes	v2 training
`vision_sft_run005_composition_rebalance`	Run-005	13	122	yes	v2 rebal
`vision_sft_holdout_run004_random`	Run-004	13	100	no	v2 hold

The v2 training recipe combines Run-003 (220 reviewed tasks) with Run-005 (122 reviewed tasks, collected specifically to rebalance under-represented fact states) and applies Run-005 twice (hereafter Run-005×2) as a moderate oversampling strategy. Bilingual export (EN + ZH) of Run-003 (220×2 = 440 rows) and Run-005×2 (122×4 = 488 rows) yields a combined training set of **928 rows**. The v2 holdout sets—Run-002 newfacts (50 samples) and Run-004 random (100 samples)—are English-only and have been verified to contain **zero exact overlap** with the training data.

6.1.2 Qwen3.5 8-Fact V1 Baseline

The earliest complete benchmark line uses an 8-fact LoRA adapter trained on the Run-001 dataset (180 tasks) and evaluated on the Run-002 holdout set (50 samples). This line predates the 13-fact ontology and serves primarily as a historical reference point; it is included here to establish the performance trajectory but may be more appropriately placed in the appendix.

Table 6.2 shows a substantial improvement in fact-level metrics: fact accuracy rises from 0.7600 to 0.9150, and seen F_1 nearly doubles from 0.4714 to 0.8380. However, the critical false positive count increases from 13 to 15, indicating that the 8-fact v1 adapter did not suppress hallucination on high-risk facts (facts whose false-positive assertions could mislead the tutoring system into skipping required steps). The sample exact match rate, at 0.36 for the LoRA adapter, shows that fully correct extraction of all eight facts within a single screenshot remained challenging.

Table 6.2: 8-fact v1 benchmark: Qwen3.5-9B Base vs. LoRA-v1 on Run-002 holdout (50 samples, EN).

Metric	Base	LoRA-v1	Δ
JSON valid rate	1.0	1.0	—
Schema valid rate	1.0	1.0	—
Fact accuracy	0.7600	0.9150	+0.1550
Macro F_1	0.4241	0.5847	+0.1605
Seen F_1	0.4714	0.8380	+0.3666
Sample exact match	0.06	0.36	+0.30
Critical false positives	13	15	+2

6.1.3 Qwen3.5 13-Fact V2 Results

The primary technical result of this work is the performance of the Qwen3.5-9B LoRA adapter trained on the Run-003 + Run-005×2 recipe under the 13-fact v2 ontology. The adapter was trained for 4 epochs at a learning rate of 2×10^{-4} with LoRA rank $r = 16$, $\alpha = 16$, dropout = 0.0, effective batch size 4, and maximum sequence length 4096. Training converged in 6419 seconds with a final training loss of 0.2156.

Table 6.3 reports results on two held-out evaluation sets: the Run-002 newfacts holdout (50 samples) and the Run-004 random holdout (100 samples). Both holdouts are English-only and have zero exact overlap with the training data.

Table 6.3: 13-fact v2 benchmark: Qwen3.5-9B Base vs. LoRA (Run-003 + Run-005×2) on two holdout sets.

	Run-002 newfacts (50)			Run-004 random (100)		
Metric	**Base**	**LoRA**	**Δ**	**Base**	**LoRA**	**Δ**
JSON valid	1.0	1.0	—	0.96	1.0	+0.04
Schema valid	1.0	1.0	—	0.96	1.0	+0.04
Fact accuracy	0.8677	**0.9908**	+0.1231	0.8038	**0.9931**	+0.1892
Macro F_1	0.4513	**0.6515**	+0.2002	0.4393	**0.6100**	+0.1707
Seen F_1	0.5022	**0.9648**	+0.4626	0.5659	**0.9159**	+0.3500
Exact match	0.10	**0.88**	+0.78	0.03	**0.92**	+0.89
Critical FP	11	**4**	−7	70	**8**	−62

Run-002 newfacts holdout. The LoRA adapter achieves a fact accuracy of 0.9908, a seen F_1 of 0.9648, and a sample exact match rate of 0.88. This means that in 44 out of 50 samples, all 13 facts are correctly classified. The base model, in contrast, achieves a fact accuracy of 0.8677 and an exact match rate of only 0.10. The critical false positive count

drops from 11 (base) to 4 (LoRA), a reduction of 7. The three remaining error sources in the LoRA adapter are: two false negatives on `tac_page_visible` (the adapter misses the TAC page when it is present), one false positive on `fcsmc_final_go_result_visible`, one false positive on `ins_grnd_alignment_text_visible`, and two false positives on `ins_ok_text_visible`.

Run-004 random holdout. The pattern is consistent but the base model degrades more severely on this larger and more diverse holdout set. The base model produces 4 invalid JSON/schema responses (yielding a validity rate of 0.96), 70 critical false positives (nearly all concentrated on `fcsmc_intermediate_result_visible` with 60), and a sample exact match rate of only 0.03. The LoRA adapter restores perfect JSON and schema validity (1.0), raises fact accuracy to 0.9931, and achieves a sample exact match rate of 0.92 (92 out of 100 samples fully correct). The critical false positive count drops from 70 to 8, all of which are on `fcsmc_final_go_result_visible`.

Across both holdouts, the LoRA adapter demonstrates three key properties. First, it restores output format reliability: the base model occasionally produces malformed JSON or schema-invalid responses on the Run-004 holdout, while the LoRA adapter does not. Second, it dramatically improves per-fact recall, as captured by seen F_1 rising from the range 0.50–0.57 (base) to 0.92–0.96 (LoRA). Third, it reduces critical false positives by 64% (Run-002) and 89% (Run-004), though a residual set of false positives on completion-type facts (`fcsmc_final_go_result_visible`, `ins_ok_text_visible`) persists.

Comparison to 8-fact v1. Moving from the 8-fact v1 to the 13-fact v2 ontology, the LoRA adapter's fact accuracy rises from 0.9150 to 0.9908 on Run-002-family holdouts, and the sample exact match rate rises from 0.36 to 0.88. Critically, the v2 adapter reduces critical false positives (from 15 to 4 on Run-002-family) rather than increasing them, as the v1 adapter did relative to its base model. This indicates that the combination of the expanded fact ontology, the Run-003 + Run-005×2 data recipe, and the composition-rebalance strategy addressed the false-positive vulnerability that was present in the v1 line.

6.1.4 Gemma-4 13-Fact V2 Comparison

To assess whether the training recipe generalises beyond a single backbone architecture, the identical data recipe (Run-003 + Run-005×2, 928 rows, bilingual, 13-fact ontology) was applied to a second VLM backbone: `google/gemma-4-31B`. Training hyperparameters were matched wherever possible: 4 epochs, learning rate 2×10^{-4}, LoRA $r = 16$,

$\alpha = 16$, dropout = 0.0, effective batch size 4, maximum sequence length 4096. Differences include the use of `all-linear` LoRA target modules (vs. Qwen's vision+language attachment), 4-bit quantization (`load_in_4bit=true`), GPU memory utilisation of 0.95, and the `gemma-4` chat template. Training converged in 8050 seconds with a final training loss of 0.0474. Note that training loss values are not directly comparable across backbones due to differences in tokenizer behaviour, template structure, and parameterisation.

Table 6.4: 13-fact v2 benchmark: Gemma-4-31B Base vs. LoRA (Run-003 + Run-005×2) on two holdout sets.

	Run-002 newfacts (50)			**Run-004 random (100)**		
Metric	**Base**	**LoRA**	**Δ**	**Base**	**LoRA**	**Δ**
JSON valid	1.0	1.0	—	0.99	1.0	+0.01
Schema valid	1.0	1.0	—	0.99	1.0	+0.01
Fact accuracy	0.8169	0.9492	+0.1323	0.8146	0.9715	+0.1569
Macro F_1	0.4432	0.5857	+0.1425	0.4400	0.5630	+0.1229
Seen F_1	0.5128	0.8123	+0.2995	0.6332	0.7959	+0.1627
Exact match	0.04	0.44	+0.40	0.12	0.74	+0.62
Critical FP	16	**0**	−16	47	13	−34

Table 6.4 confirms that the Run-003 + Run-005×2 data recipe produces meaningful improvements for the Gemma backbone as well. On both holdouts, fact accuracy rises from the base model's range of 0.81–0.82 to 0.95–0.97 for the LoRA adapter. The most distinctive result is on the Run-002 newfacts holdout, where the Gemma LoRA adapter achieves **zero critical false positives** (down from 16 for the Gemma base model). This contrasts with the Qwen LoRA adapter, which retains 4 critical false positives on the same holdout.

However, Gemma's conservative behaviour comes at a cost in recall and exact-match performance. On Run-002, the Gemma LoRA adapter's sample exact match rate is 0.44, compared to 0.88 for Qwen LoRA. On Run-004, the gap is 0.74 (Gemma) vs. 0.92 (Qwen). Seen F_1 follows a similar pattern: 0.81 (Gemma) vs. 0.96 (Qwen) on Run-002, and 0.80 (Gemma) vs. 0.92 (Qwen) on Run-004. The Gemma base model is a stronger starting point on Run-004 (seen F_1 of 0.6332 vs. Qwen base's 0.5659), but the Qwen LoRA adapter's improvement is larger, yielding a higher absolute performance ceiling.

The practical interpretation, consistent with the backbone comparison report, is that the Qwen3.5-9B adapter remains the better choice for downstream system integration, owing to its higher overall accuracy, stronger seen F_1, and higher sample exact match

rate. Gemma's zero-critical-FP result on Run-002 is noteworthy and suggests that backbone-level differences in error style (conservative vs. assertive) are observable even under matched training data.

6.1.5 Error Analysis

To understand where the remaining errors occur and what patterns differentiate the base model from the LoRA adapter, this section provides a per-fact breakdown of the strongest configuration (Qwen LoRA on the 13-fact v2 ontology).

Fact Categories by Difficulty

The 13 visual facts can be grouped into three categories that reflect distinct levels of extraction difficulty.

Group 1: Coarse panel presence (6 facts). Facts in this group indicate whether a particular cockpit panel or display page is currently visible: `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `bit_root_page_visible`, `fcsmc_page_visible`, and `hsi_page_visible`. These are structural recognition tasks that rely on identifying large, visually distinctive panel layouts.

The Qwen LoRA adapter achieves perfect classification on all six facts on both the Run-002 newfacts and Run-004 random holdouts, with the single exception of `tac_page_visible` on Run-002 (two false negatives out of 50 samples; the TAC page is seen in 23 of 50 samples, so these represent missed detections rather than hallucinations). The base model, by contrast, fails on several of these facts: it completely misses all five occurrences of `supt_page_visible` on Run-002 (seen $F_1 = 0.0$), hallucinates five false positives on `bit_root_page_visible` on Run-002, and misses 18 of 94 occurrences of `fcs_page_visible` on Run-004 (seen recall = 0.81).

Group 2: Fine-grained state indicators (5 facts). Facts in this group capture subtle within-panel states: `fcs_page_x_marks_visible` (X-marks on the FCS page), `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible` (three progressive states of the FCS-MC test sequence), and `hsi_map_layer_visible` (whether the map overlay is active on the HSI).

The Qwen LoRA adapter achieves perfect classification on `fcs_page_x_marks_visible`, `fcsmc_intermediate_result_visible`, and `hsi_map_layer_visible` on both holdouts. One false negative occurs on `fcsmc_in_test_visible` on Run-004 (17 seen samples, 16 correctly identified). The most persistent error source is `fcsmc_final_go_result_visible`

where the adapter produces 1 false positive on Run-002 and 8 false positives on Run-004 (against 60 seen occurrences). This fact requires distinguishing a specific numeric readout within the FCS-MC panel, and the consistent false-positive pattern suggests that visually similar intermediate states are occasionally misclassified as the final GO result.

The base model's behaviour on this group is qualitatively different. On Run-004, the base model generates 60 critical false positives on `fcsmc_intermediate_result_visible` alone (against only 7 true seen occurrences), indicating a near-complete failure to distinguish this state. The base model also shows poor recall on `fcs_page_x_marks_visible` (seen $F_1 = 0.0$ on both holdouts) and `fcsmc_in_test_visible` (seen recall ≈ 0.33–0.53).

Group 3: Small-text recognition (2 facts). Facts `ins_grnd_alignment_text_visible` and `ins_ok_text_visible` require the model to recognise small textual indicators on the INS panel. These are the most fine-grained facts in the ontology and represent a near-OCR-level challenge for the 9B-parameter model.

The Qwen LoRA adapter performs strongly on `ins_grnd_alignment_text_visible`: perfect classification on Run-004 (69 seen, 31 not seen) and one false positive on Run-002 (42 seen, 7 not seen). On `ins_ok_text_visible`, the adapter produces 2 false positives on Run-002 (against 4 seen occurrences; these are samples where the OK text is not yet visible but the adapter asserts it is). On Run-004, where `ins_ok_text_visible` is never seen (0 of 100), the adapter correctly produces no false positives, achieving perfect accuracy.

The base model is effectively blind to both of these facts: seen $F_1 = 0.0$ on both holdouts for `ins_grnd_alignment_text_visible`, and seen $F_1 = 0.0$ for `ins_ok_text_visible` (though the base model also never sees it on Run-004, so the F_1 metric reports 0.0 by construction).

Error Taxonomy

Aggregating across both holdouts for the Qwen LoRA adapter, the observed errors can be classified into three patterns:

1. **Completion-fact false positives (majority).** The largest share of residual errors comes from `fcsmc_final_go_result_visible` and `ins_ok_text_visible`, where the adapter occasionally asserts a completion/finalisation state that is not yet present. These are classified as *critical* false positives in the benchmark because

they carry direct downstream risk: if the tutoring system believes a step is complete when it is not, it may prematurely advance the procedure.

2. **Isolated false negatives (minor).** Two false negatives on `tac_page_visible` (Run-002) and one on `fcsmc_in_test_visible` (Run-004) represent missed detections of states that are actually present. These are less severe from a tutoring-safety perspective (a missed detection triggers a redundant prompt rather than skipping a step) but indicate residual recall gaps.

3. **Small-text hallucination (minor).** One false positive on `ins_grnd_alignment_text_visible` (Run-002) represents the adapter falsely detecting the alignment status text. Given the small visual footprint of this indicator, this error is consistent with the expected difficulty profile.

Confusion Pattern Summary

Per-fact confusion matrices (available as charts in each benchmark directory) reveal a systematic pattern in the base model's errors: the base model is prone to *unidirectional hallucination* on facts that appear rarely in the training distribution. For instance, `fcsmc_intermediate_result_visible` appears in only 7 of 100 samples in the Run-004 holdout, yet the base model asserts it as `seen` in 60 additional samples where it is not present. The LoRA adapter eliminates this class of error almost entirely, suggesting that the composition-rebalance and oversampling strategy effectively counteracted the base model's tendency to over-predict frequently co-occurring fact states.

6.1.6 Runtime Infrastructure Readiness

Beyond the VLM benchmark, the repository contains evidence of a functional replay-based evaluation infrastructure. The replay evaluation suite at `replay_eval/fa18c_startup_v04/` defines five test cases (`batteryon_2min`, `Batteryon_enginGenoff_2min`, `fcs_reset_fcs_bit_2min`, `ins_2min`, `noop_2min`) with DCS-BIOS raw captures (`dcs_bios_raw.jsonl`) and per-frame sidecar files (`frames.jsonl`). This infrastructure enables replay of recorded DCS sessions for offline testing of the telemetry pipeline, knowledge retrieval, and procedure execution logic without requiring a live DCS instance.

The runtime data collection pipeline records structured events through the `core/event_store.py` module and indexes them under `logs/test_index/index.json`. At present, this pipeline captures telemetry snapshots, procedure step transitions, and VLM inference requests.

It does not yet implement the participant/session schema, quiz linkage, or NASA-TLX integration that would be required for a human-subject study; these are discussed as planned upgrades in Section 6.2.3.

6.2 Planned Runtime Data Collection and Human-Subject Evaluation

The original research vision of this thesis, as articulated in the research proposal, includes a controlled human-subject evaluation of the grounded tutoring system in a DCS procedural learning context. This evaluation has not yet been conducted. The following subsections describe the planned evaluation design as it was formulated in the proposal, together with an assessment of the upgrades required to transition the current runtime infrastructure into a data-collection-ready state. All statements in this section represent future work and are included for transparency about the intended trajectory of the research.

6.2.1 Original Evaluation Vision

The proposal envisions a between-subjects comparison of three conditions:

1. **Video + notes baseline.** Learners study a pre-recorded video walkthrough and written checklist before and during the cold-start procedure, without any integrated tutoring system.

2. **Ungrounded text-only LLM.** A language model provides step-by-step guidance via text, but without access to real-time simulator state, retrieved manual excerpts, or visual fact extraction.

3. **Grounded RAG + LLM tutor.** The full system described in Chapters 3, 4, and 5: telemetry-grounded, retrieval-augmented, with structured visual fact extraction and overlay execution.

The original target platform is the Meta Quest 3 with DCS-VR, providing in-headset guidance through step cards or overlay annotations. The VR runtime code path is implemented; experimental validation through human-subject study is the next step before this evaluation can proceed. Target sample size is approximately $n = 5$–10 participants per condition, drawn from a beginner-to-intermediate population with limited prior F/A-18C cold-start experience.

6.2.2 Planned Evaluation Metrics

The proposal specifies a multi-dimensional evaluation framework with the following planned metrics:

- **Immediate performance:** Procedure completion rate, procedural error rate, total task time, and dead-end count (number of times the participant becomes stuck and requires external intervention).
- **Workload:** NASA Task Load Index (NASA-TLX) administered immediately after the training session.
- **Learning:** Pre-test and post-test quiz scores measuring declarative knowledge of the cold-start procedure.
- **Retention:** A delayed post-test administered 2–4 weeks after the training session to assess knowledge retention.
- **Transfer:** Performance on a variant procedure (e.g., a modified cold-start sequence) to assess whether skills generalise beyond the trained scenario.

All of these metrics remain in the planning stage; no participant-level data has been collected.

6.2.3 Required Runtime Data Collection Upgrades

To support the planned human-subject evaluation, the current runtime data collection infrastructure requires several specific upgrades:

1. **Participant and session schema.** The event store must be extended with a structured schema for participant demographics (anonymised ID, prior experience level, condition assignment), session metadata (date, duration, DCS version), and trial-level annotations.
2. **NASA-TLX integration.** A post-session instrument for administering and recording NASA-TLX responses must be integrated into the runtime, either as an in-application form or as a linked external survey with session-level traceability.
3. **Quiz linkage.** Pre-test and post-test quiz responses must be linked to participant and session identifiers to enable within-subject learning gain analysis.

4. **Anonymisation pipeline.** All participant-identifiable data must pass through an anonymisation step before storage, in compliance with standard research ethics requirements for human-subject data.

5. **VR experimental validation.** The VR runtime code path has been implemented but must be validated with human subjects, including evaluation of in-headset overlay rendering and interaction handling under study conditions.

None of these upgrades have been implemented at the time of writing.

6.2.4 Planned Experiment Design

The planned experiment follows a standard pre-test / training / post-test / retention design:

1. **Recruitment and screening.** Participants are recruited from a population with limited F/A-18C experience. A screening questionnaire establishes baseline DCS familiarity and assigns participants to conditions using stratified randomisation.

2. **Pre-test.** A written quiz assesses declarative knowledge of the cold-start procedure before any training exposure.

3. **Familiarisation.** A brief DCS cockpit familiarisation phase ensures all participants can locate basic controls and panels, reducing variance from platform unfamiliarity.

4. **Training trial.** Participants execute the F/A-18C cold-start procedure under their assigned condition. The system records all telemetry, procedure state transitions, help requests, VLM inference calls, and (for the grounded condition) overlay execution events.

5. **Post-test and NASA-TLX.** Immediately following the training trial, participants complete the NASA-TLX instrument and a post-test quiz identical in structure to the pre-test.

6. **Retention test.** After a 2–4 week interval, participants return for a delayed post-test and, where feasible, a second cold-start trial to assess procedural retention.

7. **Transfer task (optional).** A variant procedure is administered to assess the generality of learned skills.

This design has been specified at the proposal level but has not been piloted or executed.

6.2.5 Known Threats to Validity

Several threats to validity can be identified at the design stage, prior to any data collection:

1. **VR code path not yet validated with human subjects.** The VR runtime code path has been implemented but has not been validated through controlled human-subject experiments. Results obtained in either platform may not generalise to the other without additional validation.

2. **Benchmark performance does not imply learning outcomes.** The VLM benchmark results reported in Section 6.1 demonstrate that the LoRA adapter extracts visual facts accurately. However, accurate visual fact extraction is a necessary but not sufficient condition for improving human learning outcomes. A human-subject study is required to establish whether the fact extraction performance translates into measurable procedural learning gains.

3. **Small sample size.** The planned sample size of $n = 5$–10 per condition is small by the standards of controlled human-subject research and limits statistical power for detecting between-condition differences, particularly for noisy metrics such as completion time and error rate.

4. **Beginner population generalisability.** The planned participant pool consists of individuals with limited DCS experience. Results may not generalise to experienced pilots or to other simulation platforms.

5. **Single procedure.** The evaluation is designed around a single procedure (F/A-18C cold start). Whether the tutoring approach transfers to other procedural tasks within DCS or to other domains remains an open question.

6. **Experimenter effects.** In a small-n, single-site study, experimenter behaviour during familiarisation and intervention can introduce systematic bias that is difficult to separate from condition effects.

These threats do not invalidate the planned evaluation design, but they do bound the scope of conclusions that can be drawn from its eventual results. Addressing them

through careful protocol design, pre-registration of analysis plans, and explicit acknowledgement of limitations will be essential when the study proceeds.

In summary, this chapter has presented (a) technical validation results confirming that the LoRA-fine-tuned VLM adapter substantially and consistently outperforms the base model across two independent holdout sets under a 13-fact visual ontology, with the Qwen3.5-9B backbone achieving the strongest overall results; (b) evidence that the training recipe generalises to a second backbone architecture (Gemma-4-31B), though with a different precision–recall trade-off; (c) a per-fact error analysis identifying completion-fact false positives as the primary residual failure mode; and (d) a structured description of the planned human-subject evaluation, including its design, required infrastructure upgrades, and known threats to validity, all of which remain future work.DSMLparameter>

7 Discussion

This chapter interprets the results presented in Chapter 6 and situates them within the larger trajectory of the thesis. Section 7.1 explains why the project evolved from the original VR-first proposal toward the telemetry- and VLM-heavy implementation reported here. Section 7.2 interprets what the benchmark results do and do not demonstrate. Section 7.3 draws design lessons from the two-generation evolution of the visual fact ontology. Section 7.4 enumerates the known limitations of the current system and experimental evidence. Section 7.5 outlines directions for future work that follow from these limitations.

7.1 Evolution from Proposal to Implemented System

The system described in this thesis differs substantially from the system envisioned in the original research proposal. This section interprets those differences not as a failure of execution but as a reframing that produced a stronger technical baseline for the eventual thesis objectives.

7.1.1 Why the Shift Occurred

The original proposal envisioned a VR-first tutoring system: Meta Quest 3 deployment, in-headset step cards, gaze and hand interaction, and a between-subjects human evaluation as the primary source of evidence. The current implementation is instead a desktop/DCS runtime with telemetry grounding, structured help generation, and a VLM visual fact extraction pipeline.

This shift occurred for three interrelated reasons. First, building a reliable telemetry-grounded runtime — with DCS-BIOS ingestion, delta sanitisation, variable resolution, knowledge retrieval, and deterministic fallback — proved to be a substantial engineering prerequisite that could not be skipped. Attempting to deploy on Quest 3 before the telemetry pipeline and procedure engine were stable would have produced an unreliable prototype unsuitable for a controlled experiment. Second, during the development

of this prerequisite infrastructure, it became clear that several critical procedure steps (S08, S15, S18) require visual confirmation of display-page states that are not exported through DCS-BIOS telemetry. This gap motivated the addition of the VLM visual fact extraction component, which subsequently expanded into a self-contained adaptation and benchmarking pipeline — a line of work that was entirely absent from the original proposal. Third, the desktop/DCS configuration provided a more controlled environment for systematic benchmarking. It enabled fully automated base-vs-LoRA comparisons on held-out data with verified zero overlap, reproducible training configurations, and per-fact confusion analysis — conditions that would have been substantially harder to achieve in a VR setting with human participants.

7.1.2 This Is a Reframing, Not a Failure

The original thesis objectives — grounded procedural tutoring in an immersive simulation environment, with human-subject evaluation of learning outcomes — remain valid. What changed is the order of operations. The current system provides a concrete technical foundation: a working runtime with automated tests, a benchmarked VLM adaptation pipeline, and a replay and logging infrastructure that can support future human-subject studies. This foundation did not exist at the time of the proposal and represents non-trivial progress toward the thesis goals.

The VLM adaptation work, in particular, has become a substantial contribution in its own right. The full pipeline — from screenshot capture through AI-assisted prelabeling, human review, bilingual SFT export, LoRA fine-tuning, and base-vs-LoRA benchmarking on independent holdout sets — constitutes a reproducible workflow for adapting general-purpose VLMs to domain-specific structured extraction tasks with modest amounts of human-reviewed data. This contribution, while not displacing the original thesis vision of a VR tutoring system, extends the scope of the thesis beyond what was proposed.

In practical terms, the current state of the project can be summarised as follows: the instrumentation, procedure representation, and perception subsystems needed for a grounded tutoring system have been built, validated through automated benchmarks, and packaged in a replayable architecture. What remains is to conduct the human-subject evaluation that will determine whether these subsystems, operating together, produce measurable improvements in procedural learning.

7.2 Interpretation of Technical Results

The benchmark results reported in Section 6.1 provide evidence about the performance of the VLM visual fact extraction pipeline. This section draws interpretive conclusions from those results, distinguishing carefully between what the benchmarks demonstrate and what they do not.

7.2.1 What the Benchmark Results Demonstrate

Small-data LoRA adaptation can substantially improve domain-specific VLM extraction. Under the 13-fact v2 ontology, the Qwen3.5-9B LoRA adapter trained on 928 rows (Run-003 + Run-005×2) achieves a fact accuracy of 0.9908 on the Run-002 newfacts holdout and 0.9931 on the Run-004 random holdout, compared to 0.8677 and 0.8038 for the base model, respectively. The seen F_1 rises from 0.50–0.57 (base) to 0.92–0.96 (LoRA). These are large effect sizes by any standard, and they are observed on holdout sets with verified zero training overlap, ruling out memorisation as an explanation.

The data recipe transfers across model backbones. The identical training recipe (Run-003 + Run-005×2, bilingual, 13-fact ontology) applied to the Gemma-4-31B backbone produces a similar pattern of substantial improvement: fact accuracy rises from 0.82 to 0.95 on Run-002 and from 0.81 to 0.97 on Run-004. This cross-backbone transfer provides evidence that the improvement is driven by the data and ontology design rather than by idiosyncratic properties of a single model architecture.

JSON and schema validity become near-perfect after training. On the Run-004 random holdout, the base model produces 4 schema-invalid or JSON-invalid responses out of 100; the LoRA adapter produces none. This is not a trivial property: in a runtime system that must parse model output to extract overlay targets and step recommendations, format reliability is a hard requirement. The fine-tuning process reliably teaches the model to conform to the expected output schema.

The error profile shifts from indiscriminate hallucination to targeted false positives on completion-type facts. The base model's dominant failure mode is unidirectional hallucination: it asserts `seen` for many facts that are not present, particularly on rare states such as `fcsmc_intermediate_result_visible` (60 false positives on Run-004 out of only 7 true occurrences). The LoRA adapter eliminates this class of error almost entirely, concentrating its residual errors on a specific subset of facts: `fcsmc_final_go_result_visible` (8 false positives on Run-004) and, to a lesser extent,

`ins_ok_text_visible` (2 false positives on Run-002). This shift from diffuse hallucination to focused, interpretable error patterns is a qualitative improvement that is not fully captured by aggregate accuracy metrics.

The benchmark protocol itself constitutes a contribution. The use of holdout sets with verified zero overlap, per-fact confusion analysis, critical false positive tracking, and automated chart generation provides a reproducible evaluation framework that can be reused for future iterations of the ontology, for different backbone models, and for other procedures.

7.2.2 What the Benchmark Results Do NOT Demonstrate

They do not demonstrate that the tutoring system improves learning. The benchmarks measure the accuracy of visual fact extraction from cockpit screenshots. Accurate fact extraction is a necessary condition for a grounded tutoring system to function correctly, but it is not a sufficient condition for improving human learning outcomes. A learner could receive perfectly accurate visual facts and still fail to learn the procedure, or could learn effectively from a system with imperfect fact extraction. The causal chain from perception accuracy to learning outcomes has not been tested.

They do not demonstrate that visual fact extraction leads to better tutoring. The benchmarks evaluate the VLM component in isolation, not the end-to-end tutoring system. Whether the availability of visual facts improves the quality, timeliness, or acceptability of the tutor's help responses relative to a telemetry-only baseline is an open question that requires a system-level evaluation.

They measure a perception subtask, not pedagogical effectiveness. The metrics reported — fact accuracy, seen F_1, sample exact match, critical false positives — are perception metrics. They answer the question "does the VLM see the cockpit correctly?" rather than "does the tutoring system help the learner?". Conflating the two would be a category error.

7.2.3 The `ins_go` to `ins_ok_text` Decomposition as a Design Lesson

The 8-fact v1 ontology included a single fact, `ins_go`, which asked the VLM to recognise whether the INS alignment had reached a visible GO state. This fact proved to be the dominant source of false positives in the v1 adapter: 13 of 50 Run-002 samples were incorrectly predicted as `seen`. The issue was not that the VLM failed to learn from the

training data, but that the target itself was poorly specified. `ins_go` was not a visual fact in the same sense as "the FCS page is visible on the left DDI"; it was a procedural completion judgement that required integrating the presence of specific text ("OK"), the absence of other text (alignment countdown), and the procedural context (which phase of the alignment the system is in). Asking a single-frame VLM to make this judgement conflated perception with reasoning.

The 13-fact v2 ontology decomposes this compound target into lower-level, visually observable facts: `ins_grnd_alignment_text_visible` and `ins_ok_text_visible` each describe a specific piece of text on the AMPCD. The procedural interpretation — whether the combination of these facts indicates that alignment is complete — is deferred to the downstream procedure engine, which has access to telemetry, procedure phase, and timing information. This decomposition was not a cosmetic change; it was the single most important design decision in the ontology evolution, and the benchmark results bear out its effectiveness: `ins_grnd_alignment_text_visible` achieves a seen F_1 of 0.99 on Run-002, and `ins_ok_text_visible` produces only 2 false positives on Run-002 and zero on Run-004.

7.3 Ontology Design Lessons

The two-generation evolution from 8-fact v1 to 13-fact v2 provides concrete design lessons that may inform future visual ontology construction for procedural training systems.

7.3.1 Abstraction-Level Discipline

The central lesson is that a VLM-based visual fact extractor should be asked to report *observable visual evidence*, not *procedural conclusions*. The 8-fact v1 ontology mixed these two levels of abstraction within a single fact set: facts such as `fcs_page_visible` and `bit_root_page_visible` described visually localisable display-page states, while facts such as `ins_go` and `fcs_bit_result_visible` encoded higher-level procedural judgements. The mixed-abstraction ontology produced the predictable consequence: straightforward page-visibility facts achieved strong performance, while the compound procedural facts accounted for the majority of critical false positives.

The 13-fact v2 ontology enforces a design discipline in which every fact is a locally verifiable piece of visual evidence. This discipline has three practical consequences. First, it makes the per-fact annotation task clearer for human reviewers: deciding whether

"OK" text is visible on the AMPCD is a well-defined visual judgement; deciding whether "INS alignment is complete" is not. Second, it produces cleaner training signals, because the mapping from image content to fact label is more direct and less dependent on implicit procedural context. Third, it localises errors: when a fact is misclassified, the source of the error can be inspected in the image region that the fact describes, without needing to reason about the model's implicit understanding of procedure phase.

7.3.2 Decomposition of Compound Targets

The `ins_go` → {`ins_ok_text_visible`, `ins_grnd_alignment_text_visible`, `hsi_map_layer_visibl` decomposition is not the only example of this principle. The v1 fact `fcs_bit_result_visible` was similarly a compound target that asked the VLM to recognise a final test result rather than the visual indicators that characterise it. In v2, this is decomposed into `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, and `fcsmc_final_go_result_visi` each capturing a visually distinguishable stage in the FCS-MC test sequence. The downstream procedure engine can then interpret the sequence of these facts — together with telemetry state — to determine whether the FCS BIT is complete.

The general design principle can be stated as follows: if a human annotator cannot determine the fact label by looking at a single image without knowing the procedure step, the fact is likely too high-level and should be decomposed. This principle is operationalisable and transferable to other procedures.

7.3.3 Residual Failure Modes and Their Implications

Despite the ontology redesign, completion-type facts remain the most challenging category. On the Run-004 random holdout, all 8 residual critical false positives for the Qwen LoRA adapter are on `fcsmc_final_go_result_visible`. This fact requires the model to recognise a specific numeric readout (the final GO result) and to distinguish it from visually similar intermediate numeric readouts that appear in the same screen region.

There are two plausible interpretations of this residual difficulty. One is that it is a data coverage problem: the number of training examples in which the final GO result is visible is small, and the visual distinction between intermediate and final readouts may not be sufficiently represented. The other is that it is a fundamental resolution limitation: at the image resolution used for training and inference, the difference between an intermediate numeric value and the final GO value may be genuinely below the

discriminative threshold of the model. Disambiguating these two interpretations would require a controlled experiment that varies image resolution and training-set composition independently, which is beyond the scope of the current work.

7.3.4 Implications for Future Ontology Design

The experience of designing two generations of visual fact ontology for the F/A-18C cold-start suggests three guidelines for ontology construction in related procedural domains:

1. **Begin with a task analysis** that identifies which procedure states require visual confirmation and which can be covered by telemetry or rule-based inference. The ontology should only contain facts for states that genuinely require visual evidence.

2. **Enforce a single-frame verifiability test** during ontology design: every fact should be evaluable from a single cockpit image without procedural context. Facts that fail this test should be decomposed or deferred to downstream reasoning.

3. **Collect composition-rebalanced data** early in the process, particularly hard negatives that expose the boundary between visually similar but procedurally distinct states. The Run-005 composition rebalance was added specifically to address under-represented fact states, and its inclusion in the training recipe was critical to the reduction in hallucination.

7.4 Limitations

The following limitations bound the scope of conclusions that can be drawn from the present work. They are stated explicitly so that the contribution can be evaluated within its proper scope and so that future work can address them systematically.

7.4.1 Limitations of the Experimental Setup

Single procedure. All experiments, benchmarks, and dataset curation efforts focus on a single procedure: the F/A-18C cold-start. There is no evidence that the 13-fact ontology, the training recipe, or the LoRA adaptation approach generalises to other procedures (e.g., taxi, takeoff, landing, or emergency procedures in DCS, or procedural tasks in other simulators). The ontology design principles articulated in Section 7.3 are plausible but untested beyond the cold-start domain.

Single composite-panel layout. The visual fact extractor is trained and evaluated on a fixed three-display composite panel (left DDI, AMPCD, right DDI). Cockpit configurations that differ in the number, arrangement, or type of displays (e.g., the F-16C with two MFDs, or the A-10C with a different instrument layout) would require retraining with new data under a new layout description. The current models have not been tested on such configurations.

Small dataset sizes. The largest training configuration uses 928 rows (Run-003 + Run-005×2, bilingual). This is small by comparison with typical VLM fine-tuning datasets, which often use tens or hundreds of thousands of examples. TODO:CITE for comparison with dataset sizes reported in related VLM fine-tuning work (e.g., LLaVA instruction tuning, domain-specific VQA fine-tuning). While the results demonstrate that 928 rows can produce substantial improvements, they do not establish the upper bound of what is achievable with more data. Conversely, the results do not establish the minimum data requirement; it is possible that comparable performance could be achieved with fewer examples, which would be practically relevant for domains where annotated data is scarce.

Holdout sets from the same aircraft and cockpit generation. The holdout sets (Run-002 and Run-004) are independent sessions with verified zero overlap with the training data, but they are drawn from the same aircraft type, the same cockpit configuration, and the same rendering engine (DCS World). The benchmarks therefore measure *within-domain* generalisation (new sessions for the same cockpit), not *cross-domain* generalisation (new cockpit types or new simulators).

7.4.2 Limitations of the System

VR code path not yet validated with human subjects. The VR runtime code path is implemented (VR_MIRROR, tutor text display, composite-panel capture in VR mode) but has not been validated through controlled human-subject experiments. In-headset interaction, overlay rendering in VR, and latency characteristics under study conditions have not been tested with participants.

No human-subject evaluation. This is the most significant limitation of the current work. The benchmarks demonstrate that the VLM adapter extracts visual facts accurately on held-out data, but they provide no evidence about whether the full tutoring system — with or without visual fact extraction — improves any human-relevant outcome: learning, retention, transfer, workload, or procedural error rate. A human-subject study is required to establish the pedagogical value of the system, and the current

work does not include such a study.

The benchmark measures fact extraction, not tutoring quality. Even within the technical evaluation, the metrics reported are perception metrics computed on isolated single-frame fact extraction. They do not measure the quality, usefulness, or timeliness of the tutor's help responses, the correctness of overlay targets, or the coherence of multi-turn help sequences. An end-to-end evaluation of the full help cycle — from simulator observation through tutoring output — has not been conducted.

7.4.3 Limitations of the VLM Adaptation Results

Residual critical false positives on completion-type facts. As discussed in Section 7.3, the Qwen LoRA adapter retains critical false positives on `fcsmc_final_go_result_visible` (1 on Run-002, 8 on Run-004). In a runtime deployment, each such false positive carries the risk that the tutoring system incorrectly concludes that the FCS-MC test is complete and advances the procedure prematurely. Mitigation strategies — such as the sticky-fact TTL mechanism and the requirement for multiple confirming observations — exist but have not been evaluated end-to-end.

Limited model diversity. Two backbone models were evaluated (Qwen3.5-9B and Gemma-4-31B), both with LoRA fine-tuning using the Unsloth + PEFT + TRL stack. The results do not generalise to other model architectures (e.g., LLaVA-style models, InternVL), other fine-tuning methods (e.g., full fine-tuning, QLoRA with different rank configurations), or other training frameworks.

No ablation of training components. The training recipe combines several design choices — bilingual SFT, Run-005×2 oversampling, composition rebalancing, LoRA rank and alpha settings — without isolating the contribution of each. It is therefore unclear which components are necessary and which are incidental. A systematic ablation study would clarify the design space but has not been conducted.

7.5 Future Work

The limitations enumerated above define a concrete research agenda. This section outlines the principal directions for future work, ordered from the most immediate technical extensions to the longer-term thesis objectives.

7.5.1 VR Runtime Integration

The most direct path toward the original thesis vision is the experimental validation of the VR runtime code path with human subjects. The VR code path is already implemented (VR_MIRROR configuration, tutor text UDP injection, composite-panel capture in VR mode); what remains is to validate the telemetry pipeline and VLM inference under VR rendering loads with study participants, and to adapt the help trigger mechanism from desktop hotkey to VR controller or gaze-based interaction. The modular architecture described in Chapter 3 is designed to support this transition: the core tutoring logic, procedure engine, and VLM pipeline are simulator-platform-agnostic, and only the input/output adapters require modification.

7.5.2 Human-Subject Evaluation

The planned between-subjects evaluation described in Section 6.2 remains the central empirical objective of the thesis. Conducting this study requires completing the infrastructure upgrades enumerated in Section 6.2.3 — participant/session schema, NASA-TLX integration, quiz linkage, and anonymisation — and recruiting a sufficient participant sample. Given the small planned sample size (n = 5–10 per condition), the study should be designed and reported with attention to effect sizes and confidence intervals rather than null-hypothesis significance testing, and the analysis plan should be pre-registered to strengthen the credibility of the findings.

TODO:VERIFY — whether the three-condition design (video + notes vs. ungrounded text-only LLM vs. grounded tutor) is still feasible given the shift to desktop DCS, or whether a two-condition comparison (grounded tutor vs. ungrounded baseline) is more practical for the first study.

7.5.3 Extending the Visual Fact Ontology to Other Procedures

The current 13-fact ontology is specific to the F/A-18C cold-start. Extending the approach to other procedures — within DCS (taxi, takeoff, landing, aerial refuelling, emergency procedures) or in other simulators — would test the generality of the ontology design principles proposed in Section 7.3. Each new procedure would require a task analysis to identify vision-priority steps, a new fact ontology designed under the single-frame verifiability constraint, capture and annotation of new training data, and independent holdout evaluation. Whether the data recipe (bilingual SFT, composition rebalancing,

moderate oversampling) transfers to procedurally different domains is an open empirical question.

7.5.4 Multi-Frame Temporal Reasoning

The current visual fact extraction operates on single composite-panel images. Several cockpit states of interest involve temporal dynamics that cannot be disambiguated from a single frame: countdown values on the INS alignment page, animation sequences on the FCS-MC test display, and transient warning indicators. Extending the VLM input to short frame sequences (e.g., 3–5 consecutive frames at 1–2-second intervals) could enable the model to capture these temporal cues directly. This extension would require changes to the data capture pipeline (recording frame sequences rather than isolated screenshots), the training format (video or multi-image inputs), and the benchmark protocol (temporal fact definitions). Whether current small-scale VLMs can effectively exploit temporal information for cockpit state recognition is uncertain. **TODO:VERIFY** — feasibility of multi-frame VLM input given the current 9B-parameter scale and LoRA training regime.

7.5.5 Online Adaptation from In-Situ Corrections

The current VLM adaptation pipeline is an offline process: data is captured, pre-labeled, reviewed, and used to train a LoRA adapter in a batch cycle. In a deployed tutoring system, a learner or instructor may correct the system's visual fact predictions in real time (e.g., "the OK text is not actually visible yet"). Collecting these in-situ corrections and using them for online or incremental model adaptation — without requiring a full re-labeling and re-training cycle — could enable the system to improve its extraction accuracy over the course of a single training session or across multiple learners. This direction requires addressing challenges in label reliability (learner corrections may themselves be erroneous), catastrophic forgetting (online updates should not degrade performance on previously correct facts), and evaluation (online adaptation must be benchmarked against a fixed holdout set that remains untouched by the adaptation process). **TODO:VERIFY** — whether the current Unsloth + PEFT + TRL stack supports incremental fine-tuning without full retraining.

7.5.6 System-Level End-to-End Evaluation

Beyond the component-level VLM benchmarks reported in this thesis, a system-level evaluation of the full help cycle is needed. This would measure, under replay or live conditions: the end-to-end latency from help trigger to overlay rendering; the correctness of overlay targets on vision-priority steps; the rate of deterministic fallback (when model output fails validation); and the perceived usefulness of help responses as judged by domain experts reviewing replay logs. Such an evaluation would bridge the gap between the component-level benchmarks (reported here) and the human-subject evaluation (planned), providing evidence about whether the system's components function correctly when integrated.

7.5.7 Summary of Future Directions

Table 7.1 summarises the principal future directions and their current status.

Table 7.1: Summary of future work directions.

Direction	Current status	Key challenge
VR runtime integration	VR code path implemented; not validated with human subjects	Experimental validation, latency under study conditions
Human-subject evaluation	Design specified; infrastructure upgrades pending	Recruitment, statistical power, platform choice
Ontology extension to other procedures	13-fact v2 validated for cold-start only	Task analysis, annotation effort, domain generality
Multi-frame temporal reasoning	Single-frame pipeline operational	VLM capacity, training data volume, benchmark design
Online adaptation from corrections	Not implemented	Label reliability, catastrophic forgetting, evaluation
System-level end-to-end evaluation	Not conducted	Defining meaningful system-level metrics

The work presented in this thesis provides the technical foundation for each of these directions. The architecture, data pipeline, training infrastructure, and benchmark protocol are designed to be extensible, and the lessons learned from the two-generation ontology evolution and the cross-backbone comparison provide evidence-based guidance for the next stages of the project.

8 Conclusion

This thesis set out to design, implement, and technically validate a telemetry-grounded tutoring runtime for procedural learning in a high-fidelity flight simulator. The work was motivated by the original research vision of a VR-first grounded tutoring system with human-subject evaluation, but the contributions delivered are those of a technical baseline: a functioning runtime, a fine-tuned VLM adapter with benchmark evidence, a reproducible data pipeline, and a documented plan for the next phase of evaluation. This chapter summarises what has been completed, restates the contributions, and outlines directions for future work.

8.1 Summary of Completed Work

The thesis has produced five concrete contributions, each supported by verified claims and repository artifacts.

First, a pack-driven, telemetry-grounded tutoring runtime. The SimTutor system implements a ports-and-adapters architecture in which simulator-agnostic core logic (procedure state machine, gating rule engine, scoring, knowledge retrieval) is separated from simulator-specific adapters (DCS-BIOS telemetry receiver, telemetry enrichment and delta-sanitisation pipeline, overlay action executor, vision capture trigger). The runtime orchestrates a complete help cycle: it assembles a structured context from normalised telemetry variables, delta summaries, candidate procedure steps, a deterministic step hint, an overlay-target allowlist, retrieved knowledge snippets, and visual fact observations; passes this context to an LLM for structured help response generation; validates the response against schema, allowlist, and evidence-grounding constraints; executes validated overlay actions in the cockpit; and logs every stage as versioned, timestamped events for offline replay and analysis. When no model is available or when model output fails validation, the system degrades gracefully to a deterministic rule-based fallback. All procedure definitions, gating rules, telemetry mappings, error taxonomies, overlay target mappings, and vision fact ontologies are expressed as de-

clarative YAML configuration (a procedure pack), making the system extensible to new procedures and simulators without changes to the core logic.

Second, a structured visual fact ontology and a complete VLM adaptation pipeline. The 13-fact v2 ontology decomposes cockpit display-page state into discrete, auditable propositions across three display regions (Left DDI, Right DDI, AMPCD), with mechanisms for sticky fact persistence, time-to-live semantics, and step bindings via `all_of` and `any_of` conditions. The ontology evolved from an 8-fact v1 baseline that mixed abstraction levels and suffered from high false-positive rates on completion-type facts in holdout evaluation. The adaptation pipeline is complete and reproducible: screenshot capture in DCS, pre-labeling by a large VLM, human review, bilingual (EN+ZH) SFT export, LoRA fine-tuning via Unsloth + PEFT + TRL, and automated base-vs-LoRA benchmarking. Every artifact from this pipeline is stored in the repository.

Third, benchmark evidence that small-data LoRA adaptation substantially improves cockpit VLM fact extraction. On the 13-fact v2 ontology, the Qwen3.5-9B LoRA adapter (928 bilingual training rows, Run-003 + Run-005×2) achieves fact accuracies of 0.9908 and 0.9931 on two independent holdout sets, compared to base-model accuracies of 0.8677 and 0.8038. Sample exact match rates rise from 0.10 to 0.88 and from 0.03 to 0.92. Critical false positive counts drop by 64% and 89%. These results are obtained on holdout data with verified zero exact overlap with the training set.

Fourth, a backbone comparison demonstrating that the data recipe transfers. The same training recipe applied to Gemma-4-31B yields fact accuracies of 0.9492 and 0.9715 (base: 0.8169 and 0.8146), confirming that the data recipe, not a backbone-specific artefact, drives the improvement. The Gemma adapter achieves zero critical false positives on one holdout, revealing a backbone-level difference in error style (conservative vs. assertive) that is relevant for safety-critical deployment decisions.

Fifth, a documented evaluation plan for future human-subject studies. The thesis specifies a three-condition between-subjects design (video-and-notes, ungrounded text-only LLM, grounded tutoring), a multi-dimensional measurement framework (completion rate, error rate, task time, NASA-TLX, pre/post quizzes, retention, transfer), and a catalogue of required infrastructure upgrades and known threats to validity. This plan remains at the design stage; no participant data has been collected.

What the system is. The SimTutor system, as delivered, is a working desktop/DCS and VR (Quest 3) tutoring runtime with automated tests, a fine-tuned VLM adapter

with benchmark evidence, a replay and scoring infrastructure for offline analysis, and a documented extension path to human-subject evaluation. It demonstrates that a tutoring system can enforce explicit evidence grounding—every overlay target and step recommendation must be traceable to a telemetry variable, a gating rule, a retrieved document snippet, or a visual fact observation—in a working, testable implementation.

What the system is not. The VR runtime code path is implemented (VR_MIRROR, tutor text display, composite-panel capture) but has not yet been validated with human subjects. It has not been shown to improve learning outcomes, reduce workload, or accelerate procedural skill acquisition. These remain open questions for the next phase of the research programme. The current system provides the runtime, instrumentation, and benchmark infrastructure that makes addressing those questions feasible.

8.2 Future Work

The contributions of this thesis open several directions for future work, each of which builds directly on the completed technical baseline.

VR experimental validation. The most immediate extension is the experimental validation of the VR runtime code path with human subjects. The VR code path is already implemented (VR_MIRROR configuration, tutor text UDP injection, composite-panel capture in VR mode); what remains is to validate the telemetry pipeline and help cycle with study participants in the DCS-VR environment, and to integrate gaze and hand-tracking inputs for help triggering. The existing ports-and-adapters architecture supports this: the core tutoring logic is platform-agnostic, and the VR input/output adapters can be validated without changes to the core.

Human-subject evaluation. The planned evaluation design described in Section 6.2 should be executed as a controlled between-subjects study. This requires completing the runtime data collection upgrades identified in Section 6.2.3 (participant/session schema, NASA-TLX integration, quiz linkage, anonymisation pipeline), recruiting participants from a beginner-to-intermediate population, and conducting the full pre-test / training / post-test / retention protocol. Only this study can establish whether the grounded tutoring approach produces measurable improvements in procedural learning outcomes, workload, and retention relative to baseline conditions.

Ontology extension to other procedures. The 13-fact v2 ontology is specific to the F/A-18C cold-start task and its three-display composite panel layout. Extending the ontology to other procedures within DCS—such as navigation, weapon employment, or emergency checklists—would test the generality of the visual fact decomposition strategy and require new dataset capture and annotation cycles. The procedure pack format is designed to accommodate such extensions: a new pack directory with its own `vision_facts.yaml`, `vision_layout.yaml`, `step_registry.yaml`, and `telemetry_map.yaml` would define a new training scenario without changes to the pipeline tools.

Multi-frame temporal reasoning. The current VLM pipeline operates on single frame pairs (pre-trigger and trigger frames) and makes independent predictions per observation window. Incorporating temporal context across multiple frames could improve accuracy on facts that involve transient states or state transitions, such as the FCS-MC test sequence (`fcsmc_in_test_visible` → `fcsmc_final_go_result_visible`). Possible approaches include frame stacking as multi-image input to the VLM, recurrent aggregation of fact predictions over a sliding window, or a lightweight temporal model that consumes per-frame fact vectors and produces temporally consistent sequences.

Online adaptation. The current LoRA adapter is trained offline on a fixed dataset and deployed as a static model. An online adaptation loop, in which the system collects new labelled frames during live tutoring sessions (with tutor or expert verification) and periodically updates the adapter, could progressively improve accuracy on edge cases and adapt to changes in cockpit display configurations across DCS versions or aircraft variants. This would require lightweight fine-tuning infrastructure that can operate concurrently with the tutoring runtime and a quality-control mechanism for newly acquired labels.

Deterministic procedure engine enhancements. The current gating rule engine uses a v1 DSL with four operators (`var_gte`, `arg_in_range`, `flag_true`, `time_since`). Extending the DSL with additional operators (e.g., logical combinations, sequence counters, multi-variable conditions) and supporting user-defined gating functions would increase the expressiveness of procedure packs and reduce reliance on manual-step classification for edge cases.

The work completed in this thesis establishes that a telemetry-grounded, evidence-constrained tutoring runtime with structured visual fact extraction is technically feasible

and can be validated through automated benchmarks. The next phase of research must determine whether this technical capability translates into improved human learning in immersive simulation environments.

References

[1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

9 Appendix